Building Canton Apps

Invoice Factoring on a Self-Hosted Network

Vladimir Trifonov

April 22, 2026

A hands-on guide to building a multi-party application
on a self-hosted Canton network

Contents

I The Network **1**

1 Downloading Canton **2**
1.1 What We Are Building . . . 2
1.2 Prerequisites . . . 2
1.3 Download Canton . . . 3
1.4 What Is in the Box . . . 3
1.5 Set Up PostgreSQL . . . 4
1.6 Create the Project Structure . . . 5
1.7 Create the Shared Configuration . . . 5
1.8 What We Have So Far . . . 6

2 Starting the Sequencer **7**
2.1 What a Sequencer Does . . . 7
2.2 The Sequencer Configuration . . . 7
2.3 Port Plan . . . 8
2.4 Start the Sequencer . . . 9
2.5 Verify the Sequencer . . . 9
2.6 What Just Happened . . . 10
2.7 Troubleshooting . . . 10
2.8 What We Have So Far . . . 11

3 Starting the Mediator and Bootstrapping **12**
3.1 What a Mediator Does . . . 12
3.2 The Mediator Configuration . . . 12
3.3 Start the Mediator . . . 13
3.4 Running Everything in One Process . . . 13
3.5 Bootstrapping the Synchronizer . . . 15
3.6 Verify the Synchronizer . . . 16
3.7 Using a Bootstrap Script . . . 16
3.8 Troubleshooting . . . 17
3.9 What We Have So Far . . . 17

4 Starting Participant Nodes **18**
4.1 What a Participant Does . . . 18
4.2 Participant Configuration . . . 18
4.3 The Complete Network Configuration . . . 21
4.4 Update the Bootstrap Script . . . 23
4.5 Start the Complete Network . . . 24
4.6 Verify the Participants . . . 24

4.7 A Quick Ping Test . . . 25
4.8 Create a Start Script . . . 25
4.9 What We Have So Far . . . 26

5 The Canton Console **27**
5.1 Console Basics . . . 27
5.2 Health Checks . . . 27
5.3 Ping Test . . . 28
5.4 Allocating Parties . . . 28
5.5 Listing Parties . . . 29
5.6 Exploring the Network . . . 29
5.7 Uploading a DAR . . . 30
5.8 Creating a Deploy Script . . . 31
5.9 Console Tips . . . 32
5.10 Update the Bootstrap Script . . . 32
5.11 What We Have So Far . . . 33

II The Contracts **35**

6 The Invoice Contract **36**
6.1 The Invoice Project . . . 36
6.2 Modelling the Invoice . . . 37
6.3 Parties, Signatories, and Observers . . . 38
6.4 The `ensure` Clause . . . 38
6.5 The State Machine . . . 39
6.6 Testing with Daml Script . . . 39
6.7 Uploading to the Network . . . 41
6.8 What's Next . . . 41

7 The Factoring Workflow **42**
7.1 Authority, Step by Step . . . 42
7.2 The Code . . . 43
7.3 Two Things That Will Bite You . . . 44
7.4 Tests . . . 45
7.5 Deploy the New Version . . . 46
7.6 What's Next . . . 47

8 Payment Settlement **48**
8.1 Why Atomic . . . 48
8.2 The Token . . . 48
8.3 Atomic Settlement on `FactoringAgreement` . . . 49
8.4 Test the Full DvP . . . 50
8.5 Deploy . . . 51
8.6 What We Can Now Do . . . 51

9 Testing on Your Network **52**
9.1 Pointing Scripts at the Running Ledger . . . 52
9.2 The `participants.json` File . . . 52
9.3 A Live Workflow Script . . . 53
9.4 Running It . . . 54
9.5 Who Sees What . . . 54

9.6 A Repeatable Smoke Test . . . 55
9.7 What's Next . . . 55

III The Application **56**

10 The Backend **57**
10.1 Shape . . . 57
10.2 The Ledger Client . . . 58
10.3 Routes . . . 59
10.4 Running and Testing . . . 60
10.5 Why a Backend at All . . . 61

11 The Frontend **63**
11.1 Shape . . . 63
11.2 Vite Config . . . 64
11.3 The `fetch` Wrapper . . . 64
11.4 The App . . . 65
11.5 Running the Stack . . . 66
11.6 Manual Test Walk-through . . . 66
11.7 Building for Production . . . 66

12 End to End **68**
12.1 One-Command Network . . . 68
12.2 Three-Terminal Boot . . . 70
12.3 The Demo Script . . . 70
12.4 What You Have Built . . . 72

IV Production Ready **73**

13 Wallet Integration **74**
13.1 What the Token Standard Gives You . . . 74
13.2 Mapping Our Model . . . 74
13.3 Balances . . . 75
13.4 Further Reading . . . 75

14 Monitoring Your Network **76**
14.1 Turn On Metrics . . . 76
14.2 What to Watch . . . 77
14.3 Prometheus Scrape Config . . . 77
14.4 Grafana . . . 77
14.5 Structured Logging . . . 78
14.6 Alerting . . . 78

15 What's Next **79**
15.1 From DevNet to the Canton Network . . . 79
15.2 Contract Upgrades . . . 79
15.3 TLS and Auth in Production . . . 79
15.4 High Availability . . . 80
15.5 Scaling . . . 80
15.6 Observability Reading List . . . 80
15.7 What You Have Learned . . . 81

Part I

The Network

Chapter 1

Downloading Canton

By the end of this chapter you will have the Canton binary on your machine, PostgreSQL running in Docker, and a project directory ready for the app we are going to build: an invoice factoring platform with three parties.

1.1 What We Are Building

Before we touch any tools, here is the full picture. Our app has three parties:

Supplier creates invoices for goods delivered. Wants to get paid early.

Buyer confirms invoices are legitimate. Pays on the due date.

Factor buys invoices from the supplier at a discount, giving them cash now. Collects the full amount from the buyer later.

An invoice moves through these states:

```
Draft --> Confirmed --> Factored --> Paid
                   \--> Paid (direct, no factoring)
```

We will build this end to end: Canton nodes, Daml contracts, Express.js backend, React frontend. No quickstart templates. Every service started by hand so you understand exactly what each piece does.

1.2 Prerequisites

You need:

- **Docker Desktop** — for PostgreSQL (Canton's storage)
- **DPM** — the Daml Package Manager (`dpm build`, `dpm test`)
- **Java 17+** — Canton runs on the JVM
- **Node.js 18+** — for the backend and frontend
- **A terminal** and basic command-line proficiency

Verify your prerequisites:

```
java -version     # 17 or higher
node --version    # 18 or higher
docker --version  # any recent version
dpm version       # should show an installed SDK
```

1.3 Download Canton

Canton is distributed as a single archive from GitHub. We use the open-source version.

```
# Download Canton 3.4.11
curl -L -o canton-open-source-3.4.11.tar.gz \
  https://github.com/digital-asset/canton/releases/download/v3.4.11/canton-open-source-3.4.11.tar.gz

# Extract
tar xzf canton-open-source-3.4.11.tar.gz
```

Enterprise vs Open Source

Canton comes in two editions. The open-source version has everything we need for this book. The enterprise version adds features like high availability, advanced monitoring, and commercial support. The commands and configuration are identical.

Verify the download:

```
cd canton-open-source-3.4.11
./bin/canton --help
```

You should see Canton's help output with available commands. If you get a Java error, check that `java -version` returns 17 or higher.

1.4 What Is in the Box

The extracted directory has this structure:

```
canton-open-source-3.4.11/
  bin/
    canton              # the main binary
  config/
    participant.conf    # reference config: participant
    sequencer.conf      # reference config: sequencer
    mediator.conf       # reference config: mediator
    manager.conf        # reference config: topology manager
    sandbox.conf        # all-in-one for testing
    shared.conf         # shared settings (logging, etc.)
    storage/
      postgres.conf     # PostgreSQL connection settings
    tls/
      gen-test-certs.sh # generate self-signed certs
    jwt/                # JWT auth config templates
    remote/             # remote console configs
    util/
      postgres/
        db.sh           # database setup helper
        db.env          # connection variables
  lib/                  # JARs (Canton's runtime)
```

The key files:

- `bin/canton` — the single binary that runs any Canton node (participant, sequencer, mediator, or all at once)
- `config/*.conf` — reference configurations we will adapt for our network
- `config/storage/postgres.conf` — database connection template

One Binary, Many Roles

Canton uses a single binary for all node types. What role it plays is determined entirely by the configuration file you pass. The same `./bin/canton` starts a participant, a sequencer, a mediator, or all of them together. This is different from platforms like Hyperledger Fabric where each component has its own binary.

1.5 Set Up PostgreSQL

Canton needs a database for each node. We use Docker to run PostgreSQL:

```
# Create a docker-compose.yml for our project
mkdir -p ~/canton-invoice-app
cd ~/canton-invoice-app
```

Create `docker-compose.yml`:

```
# docker-compose.yml
version: "3.8"
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: canton
      POSTGRES_PASSWORD: canton
      POSTGRES_DB: canton
    ports:
      - "5432:5432"
    volumes:
      - canton-pgdata:/var/lib/postgresql/data
      - ./scripts/init-db.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  canton-pgdata:
```

Canton needs a separate database for each node. Create the init script:

```
mkdir -p scripts
```

Create `scripts/init-db.sql`:

```
-- One database per Canton node
CREATE DATABASE sequencer1;
CREATE DATABASE mediator1;
CREATE DATABASE participant_supplier;
CREATE DATABASE participant_buyer;
CREATE DATABASE participant_factor;

-- Grant access
GRANT ALL PRIVILEGES ON DATABASE sequencer1
  TO canton;
GRANT ALL PRIVILEGES ON DATABASE mediator1
  TO canton;
```

```
GRANT ALL PRIVILEGES ON DATABASE participant_supplier
  TO canton;
GRANT ALL PRIVILEGES ON DATABASE participant_buyer
  TO canton;
GRANT ALL PRIVILEGES ON DATABASE participant_factor
  TO canton;
```

Start PostgreSQL:

```
docker compose up -d postgres
```

Verify it is running:

```
docker compose ps
# Should show postgres as "running" or "healthy"

# Test the connection
psql -h localhost -U canton -d canton -c "SELECT 1"
# Enter password: canton
```

Why Separate Databases?

Each Canton node stores its own state independently. A participant stores contract data, a sequencer stores message ordering, a mediator stores confirmation state. Separate databases keep things clean and match a production deployment where each node might run on a different machine.

1.6 Create the Project Structure

Our app will live alongside the Canton binary. Set up the project:

```
cd ~/canton-invoice-app

# Create the directory structure
mkdir -p canton          # our custom configs
mkdir -p daml/daml       # Daml source
mkdir -p daml-test/daml/Test
mkdir -p backend/src/routes
mkdir -p frontend/src/{views,components}
```

Add the Canton binary to your PATH (or symlink it):

```
# Option 1: symlink
ln -s /path/to/canton-open-source-3.4.11/bin/canton \
  ~/canton-invoice-app/canton/canton

# Option 2: add to PATH in your shell config
export CANTON_HOME=/path/to/canton-open-source-3.4.11
export PATH="$CANTON_HOME/bin:$PATH"
```

Verify:

```
canton --help
# Should show Canton help output
```

1.7 Create the Shared Configuration

Every Canton node in our network shares some common settings. Create `canton/shared.conf`:

```
# canton/shared.conf
canton {
  monitoring {
    logging {
      api {
        max-method-length = 1000
        max-message-lines = 10000
      }
    }
  }
  parameters {
    non-standard-config = yes
    timeouts {
      processing {
        verify-active = 40.seconds
      }
    }
  }
}
```

This gives us reasonable logging and timeout defaults. We will build on this in the next chapters when we configure each node.

1.8 What We Have So Far

After this chapter, your directory looks like:

```
~/canton-invoice-app/
  canton/
    canton -> (symlink to binary)
    shared.conf
  daml/
    daml/                # empty, for now
  daml-test/
    daml/Test/           # empty, for now
  backend/
    src/routes/          # empty, for now
  frontend/
    src/views/           # empty, for now
    src/components/      # empty, for now
  scripts/
    init-db.sql
  docker-compose.yml
```

And you have:

- Canton binary downloaded and accessible
- PostgreSQL running with 5 databases (one per node)
- Project directory ready for configs, contracts, and code

In the next chapter, we start the first Canton service: the sequencer.

Chapter 2

Starting the Sequencer

The sequencer is the first Canton service we start. It is the backbone of a Canton synchronizer—responsible for ordering all messages between participants. Without it, nothing moves.

By the end of this chapter, you will have a running sequencer process backed by PostgreSQL.

2.1 What a Sequencer Does

When a participant submits a transaction, the sequencer:

1. Receives encrypted message envelopes from participants
2. Assigns a globally consistent timestamp to each message
3. Delivers the ordered messages to all recipients

The sequencer never sees contract data—it handles only encrypted envelopes. This is a fundamental privacy property of Canton: the infrastructure that orders transactions cannot read them.

The Sequencer Is a Post Office

Think of the sequencer as a post office that handles sealed envelopes. It stamps each envelope with a timestamp, sorts them into delivery order, and routes them to the right mailboxes. It never opens the envelopes. This is why Canton can run on shared infrastructure without compromising privacy.

2.2 The Sequencer Configuration

Create `canton/sequencer.conf`:

```
# canton/sequencer.conf
include required("shared.conf")

canton {
  sequencers {
    sequencer1 {
      storage {
        type = postgres
        config {
          dataSourceClass =
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
```

```
            serverName = "localhost"
            portNumber = 5432
            databaseName = "sequencer1"
            user = "canton"
            password = "canton"
          }
        }
      }

      public-api {
        address = "0.0.0.0"
        port = 5001
      }

      admin-api {
        address = "0.0.0.0"
        port = 5002
      }

      sequencer {
        type = BFT
      }
    }
  }
}
```

Let us walk through each section:

`include required("shared.conf")` pulls in the shared settings we created in Chapter 1 (logging, timeouts).

`canton.sequencers.sequencer1` defines a sequencer node named `sequencer1`. The name is arbitrary—you could call it anything.

`storage` connects to the `sequencer1` PostgreSQL database we created in Chapter 1.

`public-api` is the port participants use to send and receive messages. Port `5001`.

`admin-api` is the port for administrative operations (health checks, configuration). Port `5002`.

`sequencer.type = BFT` the sequencer driver type. In Canton 3.4.x, BFT is required even for single-node setups because it supports the group addressing needed for topology management. Despite the name, it does not require a multi-node BFT consensus when running a single sequencer—it simply uses PostgreSQL as its backend.

2.3 Port Plan

Before we start adding more nodes, here is the port plan for our entire network. Write this down—you will reference it throughout the book.

Node	Public/Ledger	Admin	JSON API
Sequencer	5001	5002	—
Mediator	—	5202	—
Participant (Supplier)	5011	5012	5013
Participant (Buyer)	5021	5022	5023
Participant (Factor)	5031	5032	5033

Table 2.1: Port assignments for all Canton nodes

This follows the pattern from Canton's own examples: public/ledger API on `xx01`, admin API on `xx02`, JSON API on `xx03`.

2.4 Start the Sequencer

Make sure PostgreSQL is running:

```
docker compose up -d postgres
```

Start the sequencer:

```
cd ~/canton-invoice-app
canton daemon -c canton/sequencer.conf
```

Foreground vs Daemon

The `daemon` flag runs Canton in the background without an interactive console. For development, you might prefer running without `daemon` to get the interactive REPL:

```
canton -c canton/sequencer.conf
```

This gives you a `@` prompt where you can type console commands. We will use the interactive mode more in Chapter 5.

You will see some JVM warnings about deprecated methods—these are harmless and can be ignored. In daemon mode, Canton logs to `log/canton.log` instead of the terminal. Check the log:

```
tail -20 log/canton.log
```

You should see lines like:

```
Starting Canton version 3.4.11 ...
Starting sequencer sequencer1
Sequencer sequencer1 has been initialized
```

The sequencer is now listening on port 5001 (public API) and port 5002 (admin API).

JVM Warnings

The `sun.misc.Unsafe::objectFieldOffset` and `java.lang.System::loadLibrary` warnings come from Canton's dependencies (Pekko, Conscrypt). They are safe to ignore. They will disappear in future JVM/library versions.

2.5 Verify the Sequencer

Open a new terminal. Check that the ports are listening:

```
# Check the sequencer ports are open
lsof -i :5001 | head -3   # public API (gRPC)
lsof -i :5002 | head -3   # admin API
# Both should show a java process
```

gRPC, Not HTTP

The sequencer's public API (port 5001) speaks gRPC (HTTP/2), not plain HTTP. A regular `curl` request will fail with a "Transport failed" error in the logs. This is normal. To interact with the sequencer, use the Canton Console (Chapter 5) or a gRPC client, not curl.

Check the logs to confirm startup:

```
tail -20 log/canton.log | grep -i "started\|running"
# Should show: "Successfully started all nodes"
# and: "Canton started"
```

Check the database. The sequencer should have created its schema:

```
psql -h localhost -U canton -d sequencer1 \
  -c "\dt" | head -10
# Should show Canton-managed tables
```

Canton Manages Its Own Schema

You never create tables manually. Canton creates, manages, and migrates its database schema automatically on startup. When you upgrade Canton, it handles schema migrations. This is true for all node types: sequencers, mediators, and participants.

2.6 What Just Happened

When the sequencer started, it:

1. Connected to the `sequencer1` PostgreSQL database
2. Created its schema (tables for message ordering, state tracking)
3. Generated its cryptographic identity (signing and encryption keys)
4. Started listening on ports 5001 (public) and 5002 (admin)
5. Began waiting for a synchronizer to be bootstrapped

The sequencer is running but not yet useful—it is not part of a synchronizer yet. A synchronizer needs both a sequencer AND a mediator. We start the mediator in the next chapter, then bootstrap them together.

2.7 Troubleshooting

"Connection refused" on port 5432: PostgreSQL is not running. Check `docker compose ps` and `docker compose up -d postgres`.

"Database sequencer1 does not exist": The `init-db.sql` script did not run. This happens if the PostgreSQL data volume already existed before you added the init script. Fix:

```
docker compose down -v   # removes volume
docker compose up -d postgres  # recreates with init
```

"Address already in use" on port 5001: Something else is using that port. Check with `lsof -i :5001` and either stop the conflicting process or change the port in `sequencer.conf`.

Java errors: Verify `java -version` returns 17 or higher. Canton requires JDK 17+.

2.8 What We Have So Far

```
~/canton-invoice-app/
  canton/
    shared.conf          # from Chapter 1
    sequencer.conf       # NEW: sequencer config
  scripts/
    init-db.sql          # from Chapter 1
  docker-compose.yml     # from Chapter 1
```

Running processes:

- PostgreSQL (Docker, port 5432) with 5 databases
- Sequencer (port 5001 public, 5002 admin)

In the next chapter, we start the mediator and bootstrap the synchronizer—the moment our two processes become a functioning network.

Chapter 3

Starting the Mediator and Bootstrapping

The sequencer orders messages. The mediator validates them. Together they form a *synchronizer*—the coordination layer that makes Canton work. In this chapter we start the mediator, then bootstrap both into a functioning synchronizer.

3.1 What a Mediator Does

After the sequencer delivers messages, participants process them and send back confirmation responses. The mediator:

1. Collects confirmation responses from all involved participants
2. Checks that enough participants approved the transaction
3. Issues a final verdict (approved or rejected)
4. Never sees contract data—only confirmation hashes and verdicts

Sequencer + Mediator = Synchronizer

A synchronizer needs both components:

- The **sequencer** provides ordering and delivery
- The **mediator** provides validation and finality

Neither can function alone. Neither sees contract payloads. Together they enable participants to reach consensus on transactions without exposing private data to the infrastructure.

3.2 The Mediator Configuration

Create `canton/mediator.conf`:

```
# canton/mediator.conf
include required("shared.conf")

canton {
  mediators {
    mediator1 {
```

```
      storage {
        type = postgres
        config {
          dataSourceClass =
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
            serverName = "localhost"
            portNumber = 5432
            databaseName = "mediator1"
            user = "canton"
            password = "canton"
          }
        }
      }

      admin-api {
        address = "0.0.0.0"
        port = 5202
      }
    }
  }
}
```

Notice the mediator has only an `admin-api`—no public API. Participants never talk to the mediator directly. The mediator communicates exclusively through the sequencer.

3.3 Start the Mediator

Make sure PostgreSQL and the sequencer are running, then start the mediator:

```
canton daemon -c canton/mediator.conf
```

Check the logs:

```
tail -20 log/canton.log | grep -i "started\|mediator"
# Look for "Successfully started all nodes"
```

Verify the admin port is listening:

```
lsof -i :5202 | head -3
# Should show a java process
```

Separate Log Files

When running multiple Canton processes, each writes to `log/canton.log` in its own working directory. If you start both from the same directory, logs will interleave. For clarity, you can either:

- Start each process from a different directory
- Or run them all in one process (we will do this shortly)

3.4 Running Everything in One Process

Starting separate processes for sequencer and mediator works, but for local development it is simpler to run them in a single Canton process. Create a combined config:

Create `canton/network.conf`:

```
# canton/network.conf
# Runs sequencer + mediator in one process
include required("shared.conf")

canton {
  sequencers {
    sequencer1 {
      storage {
        type = postgres
        config {
          dataSourceClass =
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
            serverName = "localhost"
            portNumber = 5432
            databaseName = "sequencer1"
            user = "canton"
            password = "canton"
          }
        }
      }
      public-api {
        address = "0.0.0.0"
        port = 5001
      }
      admin-api {
        address = "0.0.0.0"
        port = 5002
      }
      sequencer {
        type = BFT
      }
    }
  }

  mediators {
    mediator1 {
      storage {
        type = postgres
        config {
          dataSourceClass =
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
            serverName = "localhost"
            portNumber = 5432
            databaseName = "mediator1"
            user = "canton"
            password = "canton"
          }
        }
      }
      admin-api {
        address = "0.0.0.0"
        port = 5202
      }
    }
  }
}
```

Stop any running Canton processes, then start the combined config:

```
# Stop previous processes (if running as daemons)
pkill -f "canton daemon" 2>/dev/null
```

```
# Start both in one process (interactive mode)
canton -c canton/network.conf
```

After JVM warnings and compilation, you should see the Canton banner and the `@` prompt. Node startup messages go to `log/canton.log`, not the terminal. The `@` prompt is the Canton interactive console—both nodes are running in the same process.

Reading the Canton console

The console is an Ammonite REPL. Two things trip up newcomers:

- **One statement per line.** The REPL evaluates each `@` line as a complete expression. Writing `val x =` with the right-hand side on the next line throws `'=' expected but eof found`. Keep each `val` on a single line, or wrap a block in `{ ... }`. Multi-line function calls are fine as long as the open parenthesis is on the first line — the REPL waits for the matching `)`.
- **Output is verbose, not the word "Success".** Health and status calls return a detailed block (`NodeStatus[...]` or `CantonStatus`) with uptime, ports, and a component list. A node is healthy when `Active: true`, every `Components:` entry shows `Ok()`, and sequencers show `SequencerHealthStatus(active = true)`.

3.5 Bootstrapping the Synchronizer

The sequencer and mediator are running, but they do not know about each other yet. Bootstrapping is the one-time process that creates a synchronizer from these components.

In the Canton console (`@` prompt), type:

```
bootstrap.synchronizer(
  synchronizerName = "invoice-network",
  sequencers = Seq(sequencer1),
  mediators = Seq(mediator1),
  synchronizerOwners = Seq(sequencer1),
  synchronizerThreshold = PositiveInt.one,
  staticSynchronizerParameters =
    StaticSynchronizerParameters
      .defaultsWithoutKMS(
        ProtocolVersion.forSynchronizer
      ),
)
```

You should see output confirming the synchronizer was created, including its unique identifier:

```
res0: SynchronizerId =
  invoice-network::1220abcd...
```

What Bootstrap Does

The `bootstrap.synchronizer` command:

1. Creates a unique synchronizer identity
2. Registers the sequencer and mediator as members

3. Establishes the initial topology (who owns what)
4. Sets protocol parameters (version, crypto schemes)

This only runs once. After bootstrapping, the synchronizer persists its state in the database. Restarting the processes picks up where they left off.

3.6 Verify the Synchronizer

Still in the console, check that the synchronizer is alive:

```
// List sequencers
sequencer1.health.status

// Check the synchronizer ID
sequencer1.synchronizer_id

// List mediators
mediator1.health.status
```

Each call returns a status block; look for `Active: true` and all `Components:` entries reading `Ok()`. The synchronizer is live and waiting for participants to connect.

3.7 Using a Bootstrap Script

Typing bootstrap commands manually works but is tedious. Canton supports bootstrap scripts—files executed automatically on startup. Create `canton/bootstrap.canton`:

```
# canton/bootstrap.canton
# Run once to set up the synchronizer

import com.digitalasset.canton.config.RequireTypes
  .PositiveInt

bootstrap.synchronizer(
  synchronizerName = "invoice-network",
  sequencers = Seq(sequencer1),
  mediators = Seq(mediator1),
  synchronizerOwners = Seq(sequencer1),
  synchronizerThreshold = PositiveInt.one,
  staticSynchronizerParameters =
    StaticSynchronizerParameters
      .defaultsWithoutKMS(
        ProtocolVersion.forSynchronizer
      ),
)
```

Run it with the `-bootstrap` flag:

```
canton \
  -c canton/network.conf \
  --bootstrap canton/bootstrap.canton
```

The bootstrap script runs once on startup, then drops you into the interactive console. On subsequent restarts (without `-bootstrap`), the synchronizer is already configured and starts automatically.

Bootstrap Is Idempotent

You can safely run `-bootstrap` on every start. If the synchronizer already exists, Canton silently skips the bootstrap step. No error, no duplicate. This means you can keep `-bootstrap` in your start command permanently.

3.8 Troubleshooting

No output from bootstrap: This is normal on subsequent starts—the synchronizer already exists and Canton skips silently.

"Mediator not reachable": The mediator must be running before bootstrapping. If using separate processes, start the mediator first. If using `network.conf`, both start together.

"ProtocolVersion not found": Make sure you are using Canton 3.4.x. Older versions use different API names.

"Bootstrap timed out": Check that both sequencer and mediator ports are listening (`lsof -i :5001` and `lsof -i :5202`).

3.9 What We Have So Far

```
~/canton-invoice-app/
  canton/
    shared.conf          # from Chapter 1
    sequencer.conf       # from Chapter 2
    mediator.conf        # NEW
    network.conf         # NEW: combined config
    bootstrap.canton     # NEW: bootstrap script
  scripts/
    init-db.sql          # from Chapter 1
  docker-compose.yml     # from Chapter 1
```

Running processes:

- PostgreSQL (Docker, port 5432)
- Canton (sequencer on 5001/5002 + mediator on 5202)
- Synchronizer "invoice-network" bootstrapped and active

The synchronizer is live. In the next chapter we add participants—the nodes that actually run Daml contracts.

Chapter 4

Starting Participant Nodes

The synchronizer is running. Now we add the nodes that actually matter to our application: participant nodes. Each participant hosts parties, runs Daml contracts, and exposes the Ledger API that applications connect to.

Our invoice factoring platform has three parties, so we need three participants:

Participant	Ledger	Admin	JSON	Hosts
participantSupplier	5011	5012	5013	Supplier party
participantBuyer	5021	5022	5023	Buyer party
participantFactor	5031	5032	5033	Factor party

Table 4.1: Participant nodes and their ports

4.1 What a Participant Does

A participant node is where the action happens:

- **Hosts parties** — each party (Supplier, Buyer, Factor) lives on a participant
- **Runs Daml contracts** — evaluates choices, validates transactions
- **Stores contract data** — the only node type that sees contract payloads
- **Exposes the Ledger API** — applications submit commands and read state through this API

Participants See Data, Infrastructure Does Not

The sequencer and mediator never see contract data. Only participants see the contracts their parties are involved in. This is Canton's sub-transaction privacy: the Supplier's participant sees the Supplier's contracts, but not the Factor's private data. The synchronizer sees only encrypted envelopes.

4.2 Participant Configuration

Each participant has the same structure with different ports and databases. Create all three configs.

`canton/participantSupplier.conf`:

```
# canton/participantSupplier.conf
include required("shared.conf")

canton {
  participants {
    participantSupplier {
      storage {
        type = postgres
        config {
          dataSourceClass =
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
            serverName = "localhost"
            portNumber = 5432
            databaseName = "participant_supplier"
            user = "canton"
            password = "canton"
          }
        }
      }

      ledger-api {
        address = "0.0.0.0"
        port = 5011
      }

      admin-api {
        address = "0.0.0.0"
        port = 5012
      }

      http-ledger-api {
        address = "0.0.0.0"
        port = 5013
      }
    }
  }
}
```

canton/participantBuyer.conf:

```
# canton/participantBuyer.conf
include required("shared.conf")

canton {
  participants {
    participantBuyer {
      storage {
        type = postgres
        config {
          dataSourceClass =
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
            serverName = "localhost"
            portNumber = 5432
            databaseName = "participant_buyer"
            user = "canton"
            password = "canton"
          }
        }
      }

      ledger-api {
```

```
        address = "0.0.0.0"
        port = 5021
      }

      admin-api {
        address = "0.0.0.0"
        port = 5022
      }

      http-ledger-api {
        address = "0.0.0.0"
        port = 5023
      }
    }
  }
}
```

canton/participantFactor.conf:

```
# canton/participantFactor.conf
include required("shared.conf")

canton {
  participants {
    participantFactor {
      storage {
        type = postgres
        config {
          dataSourceClass =
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
            serverName = "localhost"
            portNumber = 5432
            databaseName = "participant_factor"
            user = "canton"
            password = "canton"
          }
        }
      }

      ledger-api {
        address = "0.0.0.0"
        port = 5031
      }

      admin-api {
        address = "0.0.0.0"
        port = 5032
      }

      http-ledger-api {
        address = "0.0.0.0"
        port = 5033
      }
    }
  }
}
```

Three APIs Per Participant

Each participant exposes three APIs:

Ledger API (gRPC, port xx11) — the primary API for submitting transactions and reading contracts. Used by backends via gRPC clients or code generation.

Admin API (gRPC, port xx12) — for administrative operations: party allocation, DAR uploads, health checks.

JSON Ledger API (HTTP, port xx13) — an HTTP wrapper around the gRPC Ledger API. Convenient for `curl`, frontends, and quick testing.

4.3 The Complete Network Configuration

Rather than running 5 separate processes, we extend `network.conf` to include all nodes. Update `canton/network.conf`:

```
# canton/network.conf
# Complete local network: sequencer + mediator
#                         + 3 participants
include required("shared.conf")

canton {
  sequencers {
    sequencer1 {
      storage {
        type = postgres
        config {
          dataSourceClass =
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
            serverName = "localhost"
            portNumber = 5432
            databaseName = "sequencer1"
            user = "canton"
            password = "canton"
          }
        }
      }
      public-api {
        address = "0.0.0.0"
        port = 5001
      }
      admin-api {
        address = "0.0.0.0"
        port = 5002
      }
      sequencer {
        type = BFT
      }
    }
  }

  mediators {
    mediator1 {
      storage {
        type = postgres
        config {
          dataSourceClass =
```

```
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
            serverName = "localhost"
            portNumber = 5432
            databaseName = "mediator1"
            user = "canton"
            password = "canton"
          }
        }
      }
      admin-api {
        address = "0.0.0.0"
        port = 5202
      }
    }
  }

  participants {
    participantSupplier {
      storage {
        type = postgres
        config {
          dataSourceClass =
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
            serverName = "localhost"
            portNumber = 5432
            databaseName = "participant_supplier"
            user = "canton"
            password = "canton"
          }
        }
      }
      ledger-api.port = 5011
      admin-api.port = 5012
      http-ledger-api.port = 5013
    }

    participantBuyer {
      storage {
        type = postgres
        config {
          dataSourceClass =
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
            serverName = "localhost"
            portNumber = 5432
            databaseName = "participant_buyer"
            user = "canton"
            password = "canton"
          }
        }
      }
      ledger-api.port = 5021
      admin-api.port = 5022
      http-ledger-api.port = 5023
    }

    participantFactor {
      storage {
        type = postgres
        config {
          dataSourceClass =
```

```
            "org.postgresql.ds.PGSimpleDataSource"
          properties {
            serverName = "localhost"
            portNumber = 5432
            databaseName = "participant_factor"
            user = "canton"
            password = "canton"
          }
        }
      }
      ledger-api.port = 5031
      admin-api.port = 5032
      http-ledger-api.port = 5033
    }
  }
}
```

Notice the shorthand for participant APIs: `ledger-api.port = 5011` is equivalent to the full nested block. Canton binds to `0.0.0.0` by default for participants.

4.4 Update the Bootstrap Script

The bootstrap script needs to connect participants to the synchronizer. Update `canton/bootstrap.canton`:

```
# canton/bootstrap.canton
import com.digitalasset.canton.config.RequireTypes
  .PositiveInt

// Bootstrap the synchronizer (idempotent)
bootstrap.synchronizer(
  synchronizerName = "invoice-network",
  sequencers = Seq(sequencer1),
  mediators = Seq(mediator1),
  synchronizerOwners = Seq(sequencer1),
  synchronizerThreshold = PositiveInt.one,
  staticSynchronizerParameters =
    StaticSynchronizerParameters
      .defaultsWithoutKMS(
        ProtocolVersion.forSynchronizer
      ),
)

// Connect all participants and wait until active
participantSupplier.synchronizers
  .connect_local(sequencer1, "invoice-network")
utils.retry_until_true {
  participantSupplier.synchronizers
    .active("invoice-network")
}

participantBuyer.synchronizers
  .connect_local(sequencer1, "invoice-network")
utils.retry_until_true {
  participantBuyer.synchronizers
    .active("invoice-network")
}

participantFactor.synchronizers
  .connect_local(sequencer1, "invoice-network")
utils.retry_until_true {
  participantFactor.synchronizers
    .active("invoice-network")
```

```
}
```

connect_local vs connect

`connect_local` is a convenience for when the participant and sequencer run in the same Canton process. It automatically discovers the sequencer's URL and port. For remote connections (separate processes or machines), use `connect("alias", "http://host:port")` instead.

4.5 Start the Complete Network

Stop any previous Canton processes, then start everything:

```
canton \
  -c canton/network.conf \
  --bootstrap canton/bootstrap.canton
```

After some JVM warnings and compilation, you should see the Canton banner and the `@` prompt. The startup messages go to `log/canton.log`. The bootstrap script runs automatically, connecting each participant to the synchronizer.

4.6 Verify the Participants

In the Canton console:

```
// List all participants
participants.all.map(_.name)
// Seq("participantSupplier",
//     "participantBuyer",
//     "participantFactor")

// Check health of each (returns a detailed status block;
// healthy = Active: true, every Components: entry Ok())
participantSupplier.health.status
participantBuyer.health.status
participantFactor.health.status

// Verify they are connected
participantSupplier.synchronizers.active(
  "invoice-network"
)
// true
```

Check the JSON Ledger API from another terminal:

```
# Supplier's JSON API
curl -s http://localhost:5013/v2/version | head -5

# Buyer's JSON API
curl -s http://localhost:5023/v2/version | head -5

# Factor's JSON API
curl -s http://localhost:5033/v2/version | head -5
```

All three should respond with version information. Unlike the sequencer's gRPC-only port, the JSON Ledger API speaks plain HTTP.

4.7 A Quick Ping Test

The ultimate test: can participants talk to each other through the synchronizer? In the console:

```
// Ping from supplier to buyer
participantSupplier.health.ping(
  participantBuyer
)
// Should return: Duration = ...

// Ping from factor to supplier
participantFactor.health.ping(
  participantSupplier
)
// Should return: Duration = ...
```

If the pings succeed, your entire network is working: participants can communicate through the sequencer and mediator.

What a Ping Tests

A `health.ping` between participants exercises the entire Canton protocol:

1. The source participant creates a ping contract
2. The transaction is sequenced through the sequencer
3. The mediator collects confirmations
4. The target participant sees the contract and responds
5. The source receives the response

If this works, contracts will work.

4.8 Create a Start Script

Wrap the startup into a reusable script. Create `scripts/start-network.sh`:

```bash
#!/bin/bash
# scripts/start-network.sh
# Start the complete Canton invoice network

set -e

echo "Starting PostgreSQL..."
docker compose up -d postgres
sleep 2

echo "Starting Canton network..."
canton \
  -c canton/network.conf \
  --bootstrap canton/bootstrap.canton
```

Make it executable:

```
chmod +x scripts/start-network.sh
```

From now on, start your network with:

```
./scripts/start-network.sh
```

4.9 What We Have So Far

```
~/canton-invoice-app/
  canton/
    shared.conf
    sequencer.conf
    mediator.conf
    participantSupplier.conf     # NEW
    participantBuyer.conf        # NEW
    participantFactor.conf       # NEW
    network.conf                  # UPDATED
    bootstrap.canton              # UPDATED
  scripts/
    init-db.sql
    start-network.sh              # NEW
  docker-compose.yml
```

Running processes (all in one Canton process):

- PostgreSQL (Docker, port 5432)
- Sequencer (port 5001/5002)
- Mediator (port 5202)
- Participant Supplier (ports 5011/5012/5013)
- Participant Buyer (ports 5021/5022/5023)
- Participant Factor (ports 5031/5032/5033)
- Synchronizer "invoice-network" with all participants connected

The complete network is running. In the next chapter we explore the Canton Console, allocate parties, and upload our first DAR.

Chapter 5

The Canton Console

Your network is running: sequencer, mediator, three participants, all connected. Now we explore it. The Canton Console is an interactive Scala REPL that lets you inspect nodes, allocate parties, upload code, and run diagnostics. Everything you do in this chapter prepares the network for the Daml contracts we write next.

5.1 Console Basics

When you start Canton in interactive mode, you get the `@` prompt:

```
canton \
  -c canton/network.conf \
  --bootstrap canton/bootstrap.canton
```

The console gives you pre-configured variables for every node defined in your config:

```
// Sequencer
sequencer1

// Mediator
mediator1

// Participants
participantSupplier
participantBuyer
participantFactor
```

These are live references to running nodes. Every command you type executes against the actual network.

5.2 Health Checks

Start by confirming everything is healthy:

```
// Overall health
health.status

// Individual nodes
participantSupplier.health.status
participantBuyer.health.status
participantFactor.health.status
sequencer1.health.status
mediator1.health.status
```

Each command returns a detailed status block. The overall `health.status` aggregates every node into a `CantonStatus` report; per-node calls return a `NodeStatus[...]` with uptime, ports, connected peers, and a component breakdown.

A node is healthy when:

- `Active: true` appears in the block (participants and mediators).
- Every entry under `Components:` reads `Ok()`.
- Sequencers show `SequencerHealthStatus(active = true)`.
- Participants list `Unhealthy synchronizers: None` and at least one entry under `Connected synchronizers`.

If any component reports anything other than `Ok()`, or `Active` is `false`, inspect the logs with `tail -50 log/canton.log`.

5.3 Ping Test

The ping test exercises the entire Canton protocol—sequencing, mediation, and confirmation:

```
// Ping between participants
participantSupplier.health.ping(participantBuyer)
// Returns: Duration = 123.millis (or similar)

participantBuyer.health.ping(participantFactor)
participantFactor.health.ping(participantSupplier)
```

If all three pings succeed, every participant can communicate through the synchronizer. This means contracts will work.

What a Ping Tests

A ping creates a temporary contract on one participant, sends it through the sequencer, has the mediator validate it, and delivers it to the target participant. It exercises the full transaction pipeline. If pings work, your network is fully operational.

5.4 Allocating Parties

Parties are the identities that own contracts and authorize actions. Each party lives on a specific participant. Allocate our three business parties:

```
// Allocate parties on their participants
// (keep each val on one line --- see Chapter 3 "Reading the Canton console")
val supplier = participantSupplier.parties.enable("Supplier")
val buyer    = participantBuyer.parties.enable("Buyer")
val factor   = participantFactor.parties.enable("Factor")
```

Check the results:

When a party is allocated the console prints a line like this:

```
supplier: PartyId = Supplier::1220e81a5d9f...
```

Evaluating the identifier again returns the same `PartyId`:

```
@ supplier
res13: PartyId = Supplier::1220e81a5d9f...
```

Each party has a display name (`Supplier`) and a unique namespace hash. The hash is derived from the participant's cryptographic key.

parties.enable vs Ledger API

In the console, `parties.enable` allocates a party and registers it with the synchronizer. You can also allocate parties via the JSON Ledger API:

```
curl -d '{"partyIdHint":"Supplier"}' \
  http://localhost:5013/v2/parties
```

Both methods produce the same result. The console is more convenient for interactive setup; the API is better for automation.

5.5 Listing Parties

See which parties are registered on each participant:

```
// All parties on the supplier's participant
participantSupplier.parties.list()

// All known parties across the network
// (each participant knows about all parties)
participantBuyer.parties.list()
```

Each participant knows about all parties in the network through topology transactions distributed by the synchronizer.

5.6 Exploring the Network

The console exposes the full network topology:

```
// List all nodes
nodes.local

// List participants
participants.all.map(_.name)
// Seq(participantSupplier,
//     participantBuyer,
//     participantFactor)

// List sequencers
sequencers.all.map(_.name)
// Seq(sequencer1)

// List mediators
mediators.all.map(_.name)
// Seq(mediator1)
```

Check synchronizer connectivity:

```
// Is participant connected?
participantSupplier.synchronizers
  .active("invoice-network")
// true

// Synchronizer ID
sequencer1.synchronizer_id
// invoice-network::1220...
```

5.7 Uploading a DAR

Before we write our own contracts in Chapter 6, let us verify that DAR uploads work. Create a minimal Daml project:

```
cd ~/canton-invoice-app
dpm new test-upload
cd test-upload
dpm build
```

`dpm new` templates default to version `0.0.1`, so the build produces `test-upload/.daml/dist/test-upload-0.0.1.dar`. Upload it to all three participants via the console:

```
// Upload to all participants
// Path is relative to the directory where canton was launched
// (canton-invoice-app/), not to the test-upload project.
val darPath = "test-upload/.daml/dist/test-upload-0.0.1.dar"

participantSupplier.dars.upload(darPath)
participantBuyer.dars.upload(darPath)
participantFactor.dars.upload(darPath)
```

No such file on upload

`dars.upload` resolves paths against Canton's working directory (the folder you ran `canton -c ...` from), not against the DAR's project. If the console prints `No such file [...]`, run `pwd` in another terminal or pass an absolute path like `/Users/you/canton-invoice-app/test-upload/.daml/dist/test-upload-0.0.1.dar`. Also confirm the DAR exists — `ls test-upload/.daml/dist/` — and that the version in `test-upload/daml.yaml` matches the filename.

Verify the upload:

```
// List uploaded packages
participantSupplier.packages.list()
  .map(_.packageId).take(3)
```

You can also upload via the JSON Ledger API. Run this from `canton-invoice-app/` (the path after `@` is relative to the current shell, not to Canton):

```
# cd ~/canton-invoice-app first
curl -X POST \
  http://localhost:5013/v2/packages \
  -H "Content-Type: application/octet-stream" \
  --data-binary \
    @test-upload/.daml/dist/test-upload-0.0.1.dar
```

An empty response body (`{}`) means the upload succeeded. If curl prints `Failed to open ...`, you are in the wrong directory — either `cd` back to `canton-invoice-app/` or use an absolute path (e.g. `@$(pwd)/.daml/dist/test-upload-0.0.1.dar` when already inside `test-upload/`).

Upload to Every Participant

Every participant that needs to process contracts from a DAR must have that DAR uploaded. If the Supplier creates an Invoice contract and the Buyer needs to exercise a choice on it, both participants need the Invoice DAR. For our app, we upload to all

three.

5.8 Creating a Deploy Script

Automate DAR uploads with a script. Create `scripts/deploy-dar.sh`:

```
#!/bin/bash
# scripts/deploy-dar.sh
# Upload a DAR to all participants
set -e

DAR_PATH=$1

list_dars() {
  local dars
  dars=$(ls daml/.daml/dist/*.dar 2>/dev/null || true)
  if [ -n "$dars" ]; then
    echo "$dars" | sed 's|^|  |'
  else
    echo "  (none - run 'dpm build' inside daml/)"
  fi
}

if [ -z "$DAR_PATH" ]; then
  echo "Usage: ./scripts/deploy-dar.sh <path.dar>"
  echo ""
  echo "Available DARs:"
  list_dars
  exit 1
fi

if [ ! -f "$DAR_PATH" ]; then
  echo "ERROR: DAR not found: $DAR_PATH"
  echo ""
  echo "Available DARs in daml/.daml/dist/:"
  list_dars
  echo ""
  echo "Tip: version in daml/daml.yaml must match the filename."
  exit 1
fi

echo "Uploading $DAR_PATH to all participants..."

for PORT in 5013 5023 5033; do
  echo "  -> localhost:$PORT"
  HTTP_CODE=$(curl -s -o /tmp/deploy-dar.out -w '%{http_code}' \
    -X POST "http://localhost:$PORT/v2/packages" \
    -H "Content-Type: application/octet-stream" \
    --data-binary @"$DAR_PATH")
  cat /tmp/deploy-dar.out
  if [ "$HTTP_CODE" != "200" ]; then
    echo ""
    echo "ERROR: participant at $PORT returned HTTP $HTTP_CODE"
    rm -f /tmp/deploy-dar.out
    exit 1
  fi
done
rm -f /tmp/deploy-dar.out

echo ""
echo "Done."
```

Make it executable:

```
chmod +x scripts/deploy-dar.sh
```

Usage. At this point the only DAR on disk is the scratch project from the previous section, so test the script with that:

```
./scripts/deploy-dar.sh \
  test-upload/.daml/dist/test-upload-0.0.1.dar
```

Later chapters introduce the real `invoice-app` project. Once it is built, the same script will upload it:

```
# After 'dpm build' inside daml/ (Chapter 6 onwards)
./scripts/deploy-dar.sh \
  daml/.daml/dist/invoice-app-1.0.0.dar
```

5.9 Console Tips

A few things that save time:

```
// Tab completion works
participantS<TAB>
// completes to participantSupplier

// Get help on any object
participantSupplier.help()

// Get help on a specific command
participantSupplier.dars.help()

// Run a Canton script file
utils.script("my-script.canton")
```

5.10 Update the Bootstrap Script

Now that we know the commands work, let us add party allocation to the bootstrap script. Update `canton/bootstrap.canton`:

```
# canton/bootstrap.canton
import com.digitalasset.canton.config.RequireTypes
  .PositiveInt
import com.digitalasset.canton.console
  .LocalParticipantReference

def enableIfMissing(
    p: LocalParticipantReference, name: String) =
  p.parties.list()
    .map(_.party)
    .find(_.toProtoPrimitive.startsWith(s"$name::"))
    .getOrElse(p.parties.enable(name))

// Bootstrap synchronizer (idempotent)
bootstrap.synchronizer(
  synchronizerName = "invoice-network",
  sequencers = Seq(sequencer1),
  mediators = Seq(mediator1),
```

```
  synchronizerOwners = Seq(sequencer1),
  synchronizerThreshold = PositiveInt.one,
  staticSynchronizerParameters =
    StaticSynchronizerParameters
      .defaultsWithoutKMS(
        ProtocolVersion.forSynchronizer
      ),
)

// Connect participants
participantSupplier.synchronizers
  .connect_local(sequencer1, "invoice-network")
utils.retry_until_true {
  participantSupplier.synchronizers
    .active("invoice-network")
}

participantBuyer.synchronizers
  .connect_local(sequencer1, "invoice-network")
utils.retry_until_true {
  participantBuyer.synchronizers
    .active("invoice-network")
}

participantFactor.synchronizers
  .connect_local(sequencer1, "invoice-network")
utils.retry_until_true {
  participantFactor.synchronizers
    .active("invoice-network")
}

// Allocate parties (idempotent across restarts)
val supplier = enableIfMissing(participantSupplier, "Supplier")
val buyer    = enableIfMissing(participantBuyer,    "Buyer")
val factor   = enableIfMissing(participantFactor,   "Factor")

println(s"Supplier: $supplier")
println(s"Buyer:    $buyer")
println(s"Factor:   $factor")
println("Network ready!")
```

Now every start gives you a fully configured network with parties ready to go.

5.11 What We Have So Far

```
~/canton-invoice-app/
  canton/
    shared.conf
    sequencer.conf
    mediator.conf
    network.conf
    bootstrap.canton         # UPDATED: +parties
  scripts/
    init-db.sql
    start-network.sh
    deploy-dar.sh            # NEW
  docker-compose.yml
```

You now have:

- A running Canton network (sequencer + mediator + 3 participants)

- Three allocated parties: Supplier, Buyer, Factor
- DAR upload working via console and HTTP API
- A deploy script for automation

Part I is complete. The network infrastructure is ready. In Part II, we write the Daml contracts that power our invoice factoring platform.

Part II

The Contracts

Chapter 6

The Invoice Contract

Part I gave you a live Canton network. Part II turns it into an application. The first ingredient is a contract template, and the first template of our platform is the `Invoice`: a promise to pay, raised by a supplier against a buyer, that moves through a short lifecycle until it is paid (or, in Chapter 7, factored).

The contract is deliberately small. We want every field and every choice to earn its place — no speculative columns, no placeholder workflows. What is here is what the next four chapters actually exercise.

6.1 The Invoice Project

Inside `canton-invoice-app/` we keep one Daml project that holds every template we write. Create it alongside the network:

```
cd ~/canton-invoice-app
mkdir -p daml && cd daml
dpm init .
```

`dpm init` drops a minimal `daml.yaml`. Rename the project to `invoice-app` and bump the version to `1.0.0` so the build produces the file name the deploy script in Chapter 5 expects:

```
# daml/daml.yaml
sdk-version: 3.4.11
name: invoice-app
source: daml
version: 1.0.0
dependencies:
  - daml-prim
  - daml-stdlib
  - daml-script
```

Create an empty source directory to match the `source: daml` setting:

```
mkdir -p daml
```

After that the layout looks like this:

```
canton-invoice-app/
  daml/
    daml.yaml
    daml/
      Invoice.daml           # we write this next
      Test/Invoice.daml      # tests, further down
```

daml-script in the same package

We keep the templates and the tests in one package for the rest of Part II because it keeps the learning loop short: one `dpm test` covers everything. Chapter 9 splits the test module into its own package so production participants do not have to vet `daml-script`.

6.2 Modelling the Invoice

Open `daml/daml/Invoice.daml` and drop in the template below. Read it once end-to-end before we dissect it — the shape matters more than any single line.

```
module Invoice where

import DA.Text qualified as T

-- | Lifecycle state of an Invoice.
-- Extended in Chapter 7 with Factored.
data InvoiceStatus
  = Draft
  | Confirmed
  | Paid
  deriving (Eq, Show)

-- | A commercial invoice raised by a supplier
-- against a buyer.
template Invoice
  with
    supplier : Party
    buyer : Party
    invoiceNumber : Text
    amount : Decimal
    currency : Text
    dueDate : Time
    status : InvoiceStatus
  where
    signatory supplier
    observer buyer

    ensure amount > 0.0
        && T.length invoiceNumber > 0
        && T.length currency == 3

    -- | Buyer accepts the invoice and
    -- promises to pay.
    choice Confirm : ContractId Invoice
      controller buyer
      do
        assertMsg
          "Only a Draft invoice can be confirmed"
          (status == Draft)
        create this with status = Confirmed

    -- | Buyer settles the invoice.
    choice Pay : ContractId Invoice
      controller buyer
      do
        assertMsg
          "Only a Confirmed invoice can be paid"
          (status == Confirmed)
        create this with status = Paid
```

```
-- | Supplier withdraws a Draft invoice.
choice Cancel : ()
  controller supplier
  do
    assertMsg
      "Only a Draft invoice can be cancelled"
      (status == Draft)
    pure ()
```

6.3 Parties, Signatories, and Observers

The `signatory` and `observer` lines decide who sees the contract and who has to authorize it.

- **Signatory: `supplier`.** The supplier authorized the invoice into existence. Archive, re-create, and exercising any choice on the contract requires the signatory's implicit authorization. Signatories also see every version of the contract.
- **Observer: `buyer`.** The buyer is disclosed the contract as soon as it is created. Observers cannot archive or create; they can only read and exercise choices whose controller is them.

We do *not* add the factor as an observer here. The factor only learns about the invoice in Chapter 7, when a `FactoringOffer` explicitly discloses it.

No contract keys in Daml 3

Readers coming from Daml 2 may reach for `key (supplier, invoiceNumber) : (Party, Text)` and `maintainer key._1`. Daml-LF 2.x drops contract keys entirely; adding them to this template produces:

```
Failure to process Daml program,
this feature is not currently supported.
Contract keys.
```

We use plain query predicates instead. `invoiceNumber` is a regular field; `queryFilter` and `PQS` look-ups (Chapter 16) let us find invoices by number. The rest of the model is unaffected — the lifecycle does not depend on key-based lookup.

6.4 The `ensure` Clause

`ensure` is evaluated on every `create`. If the predicate is false the ledger rejects the command before the contract enters the active contract set. We guard three things:

- `amount > 0.0` — a zero or negative invoice is a clerical error, not a legal debt.
- `T.length invoiceNumber > 0` — without an identifier, the supplier's accounting system cannot reconcile payments.
- `T.length currency == 3` — three-letter ISO 4217 codes (`USD`, `EUR`, `CHF`...). This keeps downstream reporting honest.

`ensure` is not a replacement for choice-level guards. It fires only at creation; state transitions use `assertMsg` inside choices.

6.5 The State Machine

`InvoiceStatus` is the whole lifecycle:

```
Draft --Confirm--> Confirmed --Pay--> Paid
  |
  +---Cancel---> (archived)
```

Each choice checks the current status with `assertMsg` and either creates a new contract in the next state (`Confirm`, `Pay`) or archives without a successor (`Cancel`). The previous contract is consumed automatically — every choice is consuming unless marked `nonconsuming`.

Archive vs. `create this with`

`create this with status = Confirmed` reads like "mutate the field". It is not. The current contract is archived, a new one is created, and the new `ContractId` is returned. This is why choice return type is `ContractId Invoice`, not `()`.

6.6 Testing with Daml Script

Writing the template is half the job. Before we upload anything we want mechanical proof that the lifecycle behaves, so every chapter in Part II ships with a Daml Script suite.

Create `daml/daml/Test/Invoice.daml`:

```
module Test.Invoice where

import Daml.Script
import DA.Date (date, Month(..))
import DA.Time (time)
import DA.Assert ((===))

import Invoice

-- | Create -> Confirm -> Pay.
testHappyPath : Script ()
testHappyPath = do
  supplier <- allocateParty "Supplier"
  buyer <- allocateParty "Buyer"
  let due = time (date 2026 Dec 31) 0 0 0

  draftCid <- submit supplier do
    createCmd Invoice with
      supplier
      buyer
      invoiceNumber = "INV-001"
      amount = 1000.00
      currency = "USD"
      dueDate = due
      status = Draft

  confirmedCid <- submit buyer do
    exerciseCmd draftCid Confirm
  paidCid <- submit buyer do
    exerciseCmd confirmedCid Pay

  Some paid <- queryContractId buyer paidCid
  paid.status === Paid

-- | Buyer cannot pay a Draft invoice.
testPayRequiresConfirmed : Script ()
```

```
testPayRequiresConfirmed = do
  supplier <- allocateParty "Supplier"
  buyer <- allocateParty "Buyer"
  let due = time (date 2026 Dec 31) 0 0 0

  draftCid <- submit supplier do
    createCmd Invoice with
      supplier
      buyer
      invoiceNumber = "INV-002"
      amount = 500.00
      currency = "USD"
      dueDate = due
      status = Draft

  submitMustFail buyer do
    exerciseCmd draftCid Pay

-- | ensure rejects a zero-amount invoice.
testEnsureAmount : Script ()
testEnsureAmount = do
  supplier <- allocateParty "Supplier"
  buyer <- allocateParty "Buyer"
  let due = time (date 2026 Dec 31) 0 0 0

  submitMustFail supplier do
    createCmd Invoice with
      supplier
      buyer
      invoiceNumber = "INV-003"
      amount = 0.00
      currency = "USD"
      dueDate = due
      status = Draft

-- | Supplier can cancel a Draft invoice.
testCancel : Script ()
testCancel = do
  supplier <- allocateParty "Supplier"
  buyer <- allocateParty "Buyer"
  let due = time (date 2026 Dec 31) 0 0 0

  draftCid <- submit supplier do
    createCmd Invoice with
      supplier
      buyer
      invoiceNumber = "INV-004"
      amount = 250.00
      currency = "USD"
      dueDate = due
      status = Draft

  submit supplier do
    exerciseCmd draftCid Cancel
```

Four scenarios: happy path, illegal `Pay` on `Draft`, `ensure` violation, supplier cancellation. Run them:

```
cd ~/canton-invoice-app/daml
dpm build
dpm test
```

The build produces `.daml/dist/invoice-app-1.0.0.dar` and the test run prints:

```
Test Summary

daml/Test/Invoice.daml:testCancel: ok,
  0 active contracts, 2 transactions.
daml/Test/Invoice.daml:testEnsureAmount: ok,
  0 active contracts, 1 transactions.
daml/Test/Invoice.daml:testHappyPath: ok,
  1 active contracts, 3 transactions.
daml/Test/Invoice.daml:testPayRequiresConfirmed: ok,
  1 active contracts, 2 transactions.
```

If any test fails, read the assertion message — our `assertMsg` strings were chosen to be useful in exactly this output.

The `daml-script` bloat warning

`dpm build` prints a warning that uploading a package that depends on `daml-script` bloats the participant package store. That is fine while we iterate; Chapter 9 moves the tests into a separate package so production uploads stay lean.

6.7 Uploading to the Network

The network you started in Chapter 4 is still running, so we can hand the DAR to every participant in one go. From `canton-invoice-app/`:

```
./scripts/deploy-dar.sh \
  daml/.daml/dist/invoice-app-1.0.0.dar
```

The script walks ports 5013, 5023, 5033 (Supplier, Buyer, Factor) and prints one `{}` per participant:

```
Uploading daml/.daml/dist/invoice-app-1.0.0.dar
  to all participants...
  -> localhost:5013
{}  -> localhost:5023
{}  -> localhost:5033
{}Done.
```

An empty JSON object per line is the success response from the JSON Ledger API. Confirm the package landed by asking a participant what it knows:

```
curl -s http://localhost:5013/v2/packages \
  | jq '.packageIds | length'
```

You should see a number in the dozens — the invoice package plus its transitive dependencies (`daml-stdlib`, `daml-prim`, a handful of runtime modules). Every participant has the same package set; contracts can flow between them without more configuration.

6.8 What's Next

We have a contract with a lifecycle, four passing scenarios, and a DAR sitting on three participants. Chapter 7 adds the factor: `FactoringOffer`, `AcceptOffer`, and `CollectPayment`, turning the two-party lifecycle into a real three-party workflow.

Chapter 7

The Factoring Workflow

Chapter 6 left us with a two-party lifecycle: supplier creates, buyer confirms and pays. A factoring platform needs a third role. The factor offers to purchase a confirmed invoice at a discount; the supplier accepts, takes cash immediately, and hands the right to collect from the buyer over to the factor. This chapter adds that dance on top of `Invoice`.

Three contracts carry the workflow:

- `FactoringOffer` — factor's public proposal, visible to the supplier.
- `FactoringAgreement` — the binding deal between supplier and factor, disclosed to the buyer so they know who to pay.
- The existing `Invoice` — archived as part of acceptance. The factor's claim lives on `FactoringAgreement`, not on the invoice.

We keep everything in `daml/daml/Factoring.daml` and bump the package version when we redeploy.

7.1 Authority, Step by Step

Before touching code, walk through who authorizes what. Canton refuses any transaction that cannot produce all required signatures, so thinking this out is cheaper than hitting a runtime error.

1. **Factor proposes.** `FactoringOffer` has `signatory factor` and `observer supplier`. The factor is the only party authorizing the creation; the supplier sees the offer because they are an observer.
2. **Supplier accepts.** `AcceptOffer` is a choice on `FactoringOffer` with `controller supplier`. Exercising the choice consumes the offer. Inside the choice body:
   - The supplier's authority (controller) archives the original `Invoice` — supplier is the invoice's signatory, so that is permitted.
   - The factor's authority flows in because factor is a signatory of the `FactoringOffer` that is being consumed. That lets us `create FactoringAgreement` with signatories `supplier` and `factor`.
3. **Buyer settles.** `SettleByBuyer` has `controller buyer`. Buyer is an observer of `FactoringAgreement`, so the contract is visible to them; controllers do not need to be signatories.

Authority flows through consumption

When a choice is exercised, the transaction carries both the controller's authority *and* the signatories' authority of the consumed contract. That is why a supplier-controlled choice can create a contract that requires the factor's signature: the factor's authority is provided by the `FactoringOffer` being consumed at the same moment.

7.2 The Code

Create `daml/daml/Factoring.daml`:

```
module Factoring where

import Invoice

data AgreementStatus
  = Active
  | Settled
  | Defaulted
  deriving (Eq, Show)

-- | A factor offers to buy a confirmed invoice
-- at a discount.
template FactoringOffer
  with
    factor : Party
    supplier : Party
    buyer : Party
    invoiceCid : ContractId Invoice
    invoiceNumber : Text
    originalAmount : Decimal
    currency : Text
    dueDate : Time
    discountRate : Decimal  -- 0.05 = 5% discount
  where
    signatory factor
    observer supplier

    ensure discountRate > 0.0
        && discountRate < 1.0
        && originalAmount > 0.0

    choice AcceptOffer : ContractId FactoringAgreement
      controller supplier
      do
        invoice <- fetch invoiceCid
        assertMsg "Invoice must be Confirmed"
          (invoice.status == Confirmed)
        assertMsg "Offer does not match invoice"
          (invoice.invoiceNumber == invoiceNumber
            && invoice.supplier == supplier
            && invoice.buyer == buyer)
        archive invoiceCid
        let advance =
              originalAmount * (1.0 - discountRate)
        create FactoringAgreement with
          supplier
          factor
          buyer
          invoiceNumber
```

```
          originalAmount
          advanceAmount = advance
          currency
          dueDate
          status = Active

    choice RejectOffer : ()
      controller supplier
      do pure ()

    choice WithdrawOffer : ()
      controller factor
      do pure ()

-- | A three-party record of the factoring deal.
template FactoringAgreement
  with
    supplier : Party
    factor : Party
    buyer : Party
    invoiceNumber : Text
    originalAmount : Decimal
    advanceAmount : Decimal
    currency : Text
    dueDate : Time
    status : AgreementStatus
  where
    signatory supplier, factor
    observer buyer

    ensure originalAmount > 0.0
        && advanceAmount > 0.0
        && advanceAmount < originalAmount

    choice SettleByBuyer : ContractId FactoringAgreement
      controller buyer
      do
        assertMsg "Agreement is not active"
          (status == Active)
        create this with status = Settled

    choice MarkDefaulted : ContractId FactoringAgreement
      controller factor
      do
        assertMsg "Agreement is not active"
          (status == Active)
        create this with status = Defaulted
```

7.3 Two Things That Will Bite You

Two behaviours in Daml 3.4 tripped me up while writing this chapter. Save yourself the time:

`agreement` is a reserved word

Daml templates historically had an `agreement "..."` clause for legal prose, and the parser still treats `agreement` as a keyword. Naming a local binding `agreement`:

```
agreement <- submit supplier do
  exerciseCmd offer AcceptOffer
```

produces the unhelpful error `parse error on input "agreement"`. Use a different name (`fagr`, `fAgr`, `offerAgr`) in tests and scripts. This is easy to miss because the error points to the declaration, not the keyword collision.

Bump the version before redeploying

Canton refuses to vet two packages with the same `name` and `version` but different hashes:

```
KNOWN_PACKAGE_VERSION: Tried to vet two packages
with the same name and version: ... (invoice-app v1.0.0)
and ... (invoice-app v1.0.0).
```

Every time you change the templates and redeploy, bump `version` in `daml/daml.yaml` (`1.0.0` → `1.1.0` for this chapter) and upload the new DAR. The old version remains vetted, so in-flight contracts do not break.

7.4 Tests

Add `daml/daml/Test/Factoring.daml`. Seven scenarios cover the complete decision tree: happy path, reject, withdraw, ensure violation, wrong state, and default.

```
module Test.Factoring where

import Daml.Script
import DA.Date (date, Month(..))
import DA.Time (time)
import DA.Assert ((===))

import Invoice
import Factoring

-- | Set up supplier, buyer, factor with a Confirmed invoice.
setupConfirmed :
  Script (Party, Party, Party, ContractId Invoice)
setupConfirmed = do
  supplier <- allocateParty "Supplier"
  buyer <- allocateParty "Buyer"
  factor <- allocateParty "Factor"
  let due = time (date 2026 Dec 31) 0 0 0
  draft <- submit supplier do
    createCmd Invoice with
      supplier
      buyer
      invoiceNumber = "INV-100"
      amount = 10000.00
      currency = "USD"
      dueDate = due
      status = Draft
  confirmed <- submit buyer do
    exerciseCmd draft Confirm
  pure (supplier, buyer, factor, confirmed)

-- | Happy path: offer -> accept -> settle.
testFactorAcceptSettle : Script ()
testFactorAcceptSettle = do
  setup <- setupConfirmed
  let (supplier, buyer, factor, invoice) = setup
  let due = time (date 2026 Dec 31) 0 0 0
```

```
  offer <- submit factor do
    createCmd FactoringOffer with
      factor
      supplier
      buyer
      invoiceCid = invoice
      invoiceNumber = "INV-100"
      originalAmount = 10000.00
      currency = "USD"
      dueDate = due
      discountRate = 0.05

  fagr <- submit supplier do
    exerciseCmd offer AcceptOffer
  settled <- submit buyer do
    exerciseCmd fagr SettleByBuyer

  Some fa <- queryContractId factor settled
  fa.status === Settled
  fa.advanceAmount === 9500.00
```

The other six scenarios in the file (in the repo alongside the chapter) exercise `RejectOffer`, `WithdrawOffer`, the `discountRate` `ensure`, rejecting acceptance of a Draft invoice, and `MarkDefaulted`. Run all eleven — four from Chapter 6, seven from here:

```
cd ~/canton-invoice-app/daml
dpm build
dpm test
```

Expect output like:

```
daml/Test/Factoring.daml:testFactorAcceptSettle:
  ok, 1 active contracts, 5 transactions.
daml/Test/Factoring.daml:testFactorRejected:
  ok, 1 active contracts, 4 transactions.
daml/Test/Factoring.daml:testFactorWithdraw:
  ok, 1 active contracts, 4 transactions.
daml/Test/Factoring.daml:testBadDiscount:
  ok, 1 active contracts, 3 transactions.
daml/Test/Factoring.daml:testAcceptRequiresConfirmed:
  ok, 2 active contracts, 3 transactions.
daml/Test/Factoring.daml:testMarkDefaulted:
  ok, 1 active contracts, 5 transactions.
daml/Test/Invoice.daml:testHappyPath:
  ok, 1 active contracts, 3 transactions.
... (4 more)
```

7.5 Deploy the New Version

Upload the `1.1.0` DAR to the running network:

```
cd ~/canton-invoice-app
./scripts/deploy-dar.sh \
  daml/.daml/dist/invoice-app-1.1.0.dar
```

Three `{}` lines signal success. The old `1.0.0` vetting remains in place, so anyone holding an `Invoice` contract from Chapter 6 can still interact with it — the new templates are purely additive.

7.6 What's Next

We now model the legal and commercial flow of factoring. What we do *not* model yet is money. `SettleByBuyer` records that the buyer paid, but no asset moves. Chapter 8 introduces `PaymentToken`, turns settlement into an atomic delivery-vs-payment, and wires payment into the factoring agreement.

Chapter 8

Payment Settlement

Chapter 7 closed the factoring workflow legally: the `FactoringAgreement` records who owes whom. But `SettleByBuyer` is a claim, not a payment — no asset moves. A real platform needs money on the ledger, and the move of money must be simultaneous with the release of the claim. This chapter adds a `PaymentToken` and settles through atomic delivery-vs-payment (DvP).

8.1 Why Atomic

Imagine a two-step settlement:

1. Buyer marks the agreement `Settled`.
2. In a separate transaction, buyer transfers payment to factor.

Between the two, the buyer is discharged without having paid. Or the factor is paid without releasing the claim. Either side can walk away. In Canton, *atomic* means one transaction — all effects commit together, or none do. Our `SettleWithToken` choice moves the token and flips the agreement state in a single authorization flow.

8.2 The Token

Create `daml/daml/Payment.daml`:

```
module Payment where

import DA.Text qualified as T

-- | Minimal fungible payment token.
template PaymentToken
  with
    issuer : Party
    owner : Party
    amount : Decimal
    currency : Text
  where
    signatory issuer
    observer owner

    ensure amount > 0.0 && T.length currency == 3

    -- | Transfer to a new owner.
    -- Both current and new owner must authorize
```

```
    -- in the same transaction.
    choice Transfer : ContractId PaymentToken
      with newOwner : Party
      controller owner, newOwner
      do
        create this with owner = newOwner
```

Two design decisions earn a note.

- **Issuer-only signatory.** Every transfer archives the current token and creates a new one. If the owner were a signatory, each transfer would need the new owner's signature at create time — doable but duplicative, because `controller owner, newOwner` already requires both. Keeping only the issuer as signatory means the token is recognisably the *same* obligation across owners, and the issuer's authority flows through archival without extra ceremony.
- **Two-party controller on `Transfer`.** Listing both `owner` and `newOwner` as controllers folds propose-accept into one transaction. You cannot push a token onto someone who has not consented; you cannot take a token from someone who has not signed. In tests, that translates to `submit (actAs [oldOwner, newOwner]) ....`

This is not the Canton Token Standard

A production stablecoin uses the official Canton Token Standard (holdings, instruments, transfer factories) — see Chapter 13. `PaymentToken` here is intentionally one template with one choice so the DvP mechanics stay legible. Everything we learn about atomic settlement transfers directly to the real standard.

8.3 Atomic Settlement on `FactoringAgreement`

Add a new choice to the `FactoringAgreement` template in `Factoring.daml`:

```
-- | Atomic delivery-vs-payment: transfer a
-- PaymentToken of the right amount from buyer to
-- factor and mark the agreement Settled, all in
-- one transaction.
choice SettleWithToken : ContractId FactoringAgreement
  with tokenCid : ContractId PaymentToken
  controller buyer
  do
    assertMsg "Agreement is not active"
      (status == Active)
    token <- fetch tokenCid
    assertMsg "Token owner must be buyer"
      (token.owner == buyer)
    assertMsg "Token amount mismatch"
      (token.amount == originalAmount)
    assertMsg "Token currency mismatch"
      (token.currency == currency)
    _ <- exercise tokenCid Transfer with
      newOwner = factor
    create this with status = Settled
```

Do not delete `SettleByBuyer` — we leave it as the "record-only" settlement for cases where payment happens off-ledger and only the state needs to move. The new choice adds the on-ledger money flow.

Where authority comes from

`SettleWithToken` has `controller buyer`. Inside the body we `exercise tokenCid Transfer with newOwner = factor`, whose controllers are `owner` (buyer) and `newOwner` (factor). Buyer's authority is the choice controller. Factor's authority comes from being a signatory of the `FactoringAgreement` being exercised — which propagates into every sub-exercise. No extra multi-party submission needed at call time? There is, because the top-level `exercise` of `SettleWithToken` still hits the `Transfer` controllers check when the transaction is submitted. In scripts that surfaces as `submit (actAs [buyer, factor]) ....`

8.4 Test the Full DvP

Add `daml/daml/Test/Payment.daml` with four scenarios. Three selected:

```
module Test.Payment where

import Daml.Script
import DA.Date (date, Month(..))
import DA.Time (time)
import DA.Assert ((===))

import Invoice
import Factoring
import Payment

-- | Issue -> transfer -> verify new owner.
testTokenTransfer : Script ()
testTokenTransfer = do
  issuer <- allocateParty "Bank"
  alice <- allocateParty "Alice"
  bob <- allocateParty "Bob"

  tokenCid <- submit issuer do
    createCmd PaymentToken with
      issuer
      owner = alice
      amount = 1000.00
      currency = "USD"

  newCid <- submit (actAs [alice, bob]) do
    exerciseCmd tokenCid Transfer with
      newOwner = bob

  Some token <- queryContractId bob newCid
  token.owner === bob
  token.amount === 1000.00

-- | Buyer settles factoring agreement atomically.
testDvP : Script ()
testDvP = do
  issuer <- allocateParty "Bank"
  supplier <- allocateParty "Supplier"
  buyer <- allocateParty "Buyer"
  factor <- allocateParty "Factor"
  let due = time (date 2026 Dec 31) 0 0 0
  -- ... build an accepted agreement (as in ch 7) ...
  -- (see the repo for the full setup)

  -- Bank issues a token to the buyer for the full amount
```

```
  token <- submit issuer do
    createCmd PaymentToken with
      issuer
      owner = buyer
      amount = 10000.00
      currency = "USD"

  -- Atomic: buyer settles + token moves to factor
  settled <- submit (actAs [buyer, factor]) do
    exerciseCmd fagr SettleWithToken with
      tokenCid = token

  Some fa <- queryContractId factor settled
  fa.status === Settled
```

The full file in `daml/daml/Test/Payment.daml` also contains `testTokenEnsure` (rejects bad currency) and `testDvPWrongAmount` (rejects a token that does not match the invoice amount).

Run:

```
cd ~/canton-invoice-app/daml
dpm build
dpm test
```

All fifteen tests should be green (four Invoice, seven Factoring, four Payment). If `testDvP` fails with an authorization error, check the `submitMulti` parties — both buyer and factor must be in the `submitMulti` list because `Transfer` needs both.

8.5 Deploy

Bump `daml/daml.yaml` from `1.1.0` to `1.2.0` and redeploy:

```
cd ~/canton-invoice-app
./scripts/deploy-dar.sh \
  daml/.daml/dist/invoice-app-1.2.0.dar
```

8.6 What We Can Now Do

With three templates (`Invoice`, `FactoringOffer`/`FactoringAgreement`, `PaymentToken`) and fifteen passing tests, the platform supports the complete commercial lifecycle of an invoice from draft to paid-through-factor. The Daml side is effectively done — further chapters in Part III layer a backend and a UI on top. Before that, Chapter 9 hardens what we have by exercising the same workflow end-to-end against the running Canton network, not just in Daml Script.

Chapter 9

Testing on Your Network

Daml Script in the IDE sandbox is fast, but the contracts you've been writing are meant for a real multi-node Canton network — the one you started in Chapters 2–4 and populated in Chapters 5–8. Before moving on to the backend, we exercise the same scripts against the live ledger, wire up a `participants.json` so multi-party submissions are routed correctly, and verify what each participant sees.

9.1 Pointing Scripts at the Running Ledger

`dpm script` can talk to any gRPC-exposed Ledger API. For a single-participant script, `-ledger-host` and `-ledger-port` are enough:

```
dpm script \
  --dar .daml/dist/invoice-app-1.2.0.dar \
  --script-name Test.OnLedger:fullWorkflow \
  --ledger-host localhost --ledger-port 5011
```

But our `fullWorkflow` submits as both `Supplier` (on port 5011) and `Buyer` (on port 5021). Hitting that against a single participant produces:

```
NO_SYNCHRONIZER_ON_WHICH_ALL_SUBMITTERS_CAN_SUBMIT:
This participant cannot submit as the given submitter
on any connected synchronizer
```

A participant can only submit on behalf of parties hosted on it. `dpm script` routes per-party through a config file.

9.2 The `participants.json` File

Create `daml/participants.json` and fill in the PartyId strings from your network. Get them from any participant's JSON API:

```
curl -s http://localhost:5013/v2/parties | jq .
```

Then write:

```
{
  "default_participant": {
    "host": "localhost", "port": 5011
  },
  "participants": {
    "supplier": { "host": "localhost", "port": 5011 },
    "buyer":    { "host": "localhost", "port": 5021 },
```

```
    "factor":   { "host": "localhost", "port": 5031 }
  },
  "party_participants": {
    "Supplier::1220f7b280be...": "supplier",
    "Buyer::1220b67d78e0...":    "buyer",
    "Factor::1220098a44bd...":   "factor"
  }
}
```

The `party_participants` map tells the runner: when the script submits on behalf of `Supplier::...`, route to the `supplier` participant (port 5011). Likewise for Buyer and Factor.

Hashes differ on your network

Your `Supplier::` hash will not match the one above — every Canton network produces its own namespace. Always copy the strings from your own `curl /v2/parties` output.

9.3 A Live Workflow Script

`daml/daml/Test/OnLedger.daml` fetches the three pre-existing parties by PartyId prefix and runs the invoice lifecycle:

```
module Test.OnLedger where

import Daml.Script
import DA.Date (date, Month(..))
import DA.Time (time)
import DA.Text qualified as T

import Invoice

fullWorkflow : Script ()
fullWorkflow = do
  parties <- listKnownParties
  let findByPrefix pfx =
        find
          (\p -> T.isPrefixOf pfx
            (partyToText p.party))
          parties
  supplier <- case findByPrefix "Supplier::" of
    Some d -> pure d.party
    None   -> allocateParty "Supplier"
  buyer <- case findByPrefix "Buyer::" of
    Some d -> pure d.party
    None   -> allocateParty "Buyer"
  let due = time (date 2027 Jun 30) 0 0 0

  draft <- submit supplier do
    createCmd Invoice with
      supplier
      buyer
      invoiceNumber = "LIVE-001"
      amount = 2500.00
      currency = "USD"
      dueDate = due
      status = Draft

  confirmed <- submit buyer do
    exerciseCmd draft Confirm
  paid <- submit buyer do
```

```
    exerciseCmd confirmed Pay

  Some final <- queryContractId buyer paid
  debug ("Final status: " <> show final.status)
```

PartyDetails in Daml 3

In Daml 2, `listKnownParties` returned records with a `displayName : Optional Text` field that held the hint you gave at allocation. Daml 3 dropped `displayName` — the record has only `party`, `isLocal`, and `localMetadata`. Match on the `party` PartyId string instead (it starts with the allocation hint).

9.4 Running It

Bump the version in `daml/daml.yaml` to `1.3.0` (we added a module), rebuild, and deploy:

```
cd ~/canton-invoice-app/daml
dpm build
cd ..
./scripts/deploy-dar.sh \
  daml/.daml/dist/invoice-app-1.3.0.dar
```

Now run the live script:

```
cd daml
dpm script \
  --dar .daml/dist/invoice-app-1.3.0.dar \
  --script-name Test.OnLedger:fullWorkflow \
  --participant-config participants.json
```

Expect output similar to:

```
[DA.Internal.Prelude]: "Created Draft:
  00812ab65de53c088a030337f350240b3d37d5ca62430...
[DA.Internal.Prelude]: "Confirmed:
  00dec328ff394ba6a8263e4a2741d05ebc9ae07f2851c...
[DA.Internal.Prelude]: "Paid:
  00c48330d67354fedfdcd174e5cee720488bc6906a15f...
[DA.Internal.Prelude]: "Final status: Paid"
```

Three contract IDs, three transactions on your sequencer, a lifecycle that started as `Draft` on the supplier's participant and ends as `Paid` queried from the buyer's.

9.5 Who Sees What

Canton's privacy model gives each participant its own copy of the contracts their hosted parties can see. Ask each participant's JSON API what it knows about parties:

```
# From supplier's participant
curl -s http://localhost:5013/v2/parties \
  | jq '.partyDetails | map(.party)'

# From buyer's
curl -s http://localhost:5023/v2/parties \
  | jq '.partyDetails | map(.party)'
```

Both lists contain all three business parties — topology is global. What differs is the contract store: the factor's participant has never heard about the `LIVE-001` invoice we just

created, because the `Invoice` template discloses only to `supplier` and `buyer`. In Chapter 10 the backend will query each participant's ACS and you will see this asymmetry directly.

9.6 A Repeatable Smoke Test

Script runs are idempotent as far as the script code is concerned, but the ledger keeps every archived contract for audit. Running `fullWorkflow` a second time creates `LIVE-001` as a fresh contract because nothing enforces uniqueness on `invoiceNumber` — the absence of contract keys means you have to validate that yourself at the application layer. We will do exactly that in Chapter 10.

Wire the script into a shell helper so the full smoke test is one command:

```
# scripts/smoke-test.sh
#!/bin/bash
set -e
cd "$(dirname "$0")/../daml"
dpm build
dpm script \
  --dar .daml/dist/invoice-app-1.3.0.dar \
  --script-name Test.OnLedger:fullWorkflow \
  --participant-config participants.json
echo "Smoke test OK"
```

Run it after any code change — if the live network accepts the transactions, your templates are compatible with what is already deployed.

9.7 What's Next

The ledger is proven. Part III turns the ledger into an application: Chapter 10 builds a small Express backend that talks to the Ledger API, Chapter 11 puts a React UI on top, and Chapter 12 walks through the entire supplier→buyer→factor flow end to end.

Part III

The Application

Chapter 10

The Backend

A ledger is not a product. Users do not type gRPC. This chapter wraps Canton's JSON Ledger API in a small Express service so the rest of the stack (UI, integrations, scripts) can talk to *HTTP* instead of navigating participant ports and template IDs directly.

The backend is deliberately thin. Every route does one of three things: read the active contract set of a participant, submit a create, or exercise a choice. The Ledger API does the heavy lifting — our job is routing, authentication, and keeping the contract vocabulary out of the frontend.

10.1 Shape

```
canton-invoice-app/
  backend/
    package.json
    src/
      ledger.js       # JSON Ledger API client
      index.js        # Express app + routes
```

```
cd ~/canton-invoice-app
mkdir -p backend/src
cd backend
```

Create `package.json`:

```
{
  "name": "canton-invoice-backend",
  "version": "1.0.0",
  "type": "module",
  "main": "src/index.js",
  "scripts": {
    "start": "node src/index.js",
    "dev": "node --watch src/index.js"
  },
  "dependencies": {
    "express": "^4.21.1"
  }
}
```

Install:

```
npm install
```

Node 18+, `fetch` built-in

We do not import a HTTP client. `fetch` is available globally on Node 18 and later, which is everything reasonable in 2026. If you are on an older runtime, add `node-fetch` to `dependencies`.

10.2 The Ledger Client

`src/ledger.js` is one class per party: the request body for every Ledger API call needs to declare `actAs` and `userId`, and routing to the right participant is a matter of which port we hit.

```
// src/ledger.js (excerpt)
const DEFAULT_USER = 'ledger-api-user';

export class LedgerClient {
  constructor({ host = 'localhost', port, party,
                userId = DEFAULT_USER }) {
    this.baseUrl = `http://${host}:${port}`;
    this.party = party;
    this.userId = userId;
  }

  async submitCommand(commandId, actAs, commands) {
    return this._post(
      '/v2/commands/submit-and-wait-for-transaction-tree',
      { commands, commandId, actAs, userId: this.userId },
    );
  }

  async create(templateId, args, commandId) {
    const result = await this.submitCommand(
      commandId,
      [this.party],
      [{ CreateCommand:
          { templateId, createArguments: args } }],
    );
    const events = Object.values(
      result.transactionTree?.eventsById ?? {},
    );
    const created = events.find((e) => e.CreatedTreeEvent);
    return created?.CreatedTreeEvent?.value;
  }

  async exercise(templateId, contractId, choice,
                 choiceArg, commandId, actAs) {
    const result = await this.submitCommand(
      commandId,
      actAs ?? [this.party],
      [{ ExerciseCommand:
          { templateId, contractId, choice,
            choiceArgument: choiceArg } }],
    );
    return result.transactionTree;
  }

  async activeContracts(templateId, { limit = 100 } = {}) {
    const end = await this._get('/v2/state/ledger-end');
    return this._post(
      `/v2/state/active-contracts?limit=${limit}`,
```

```
      {
        filter: { filtersByParty: { [this.party]: {
          cumulative: [{ identifierFilter: {
            TemplateFilter: { value: {
              templateId, includeCreatedEventBlob: false,
            }},
          }}],
        }}},
        verbose: false,
        activeAtOffset: end.offset,
      },
    );
  }
}
```

The full `_get` and `_post` helpers (straight `fetch` calls with error handling) are in the repo. Every API endpoint we need — parties, commands, ACS — is one method on this class.

#package-name over package-id

Canton 3.4 accepts two `templateId` formats: a raw package hash (`0b8d07...:Invoice:Invoice`) and a package-name reference prefixed with `#`: `#invoice-app:Invoice:Invoice`. The hash form is marked deprecated in 3.4 and will disappear. Use `#invoice-app:...` — it survives version bumps without code changes.

10.3 Routes

`src/index.js` boots, resolves the three business parties from a participant (the supplier's JSON API knows everyone), and maps each role to its port:

```
// src/index.js (excerpt)
import express from 'express';
import { LedgerClient } from './ledger.js';

const AUTH_SECRET = process.env.API_SECRET ?? 'dev-secret';

async function resolveParties() {
  const lookup = new LedgerClient({ port: 5013, party: null });
  const { partyDetails } = await lookup.listParties();
  const byPrefix = (pfx) =>
    partyDetails.find((p) => p.party.startsWith(pfx))?.party;
  return {
    supplier: byPrefix('Supplier::'),
    buyer: byPrefix('Buyer::'),
    factor: byPrefix('Factor::'),
  };
}

const PORTS = { supplier: 5013, buyer: 5023, factor: 5033 };
const T_INVOICE = '#invoice-app:Invoice:Invoice';

const parties = await resolveParties();
const clientFor = (role) =>
  new LedgerClient({ port: PORTS[role], party: parties[role] });

const app = express();
app.use(express.json());

app.use((req, res, next) => {
  if (req.path === '/health') return next();
```

```
  if (req.headers.authorization !== `Bearer ${AUTH_SECRET}`) {
    return res.status(401).json({ error: 'unauthorized' });
  }
  next();
});

app.get('/health', (_req, res) =>
  res.json({ ok: true, parties }));

app.post('/invoices', async (req, res, next) => {
  try {
    const { invoiceNumber, amount, currency, dueDate }
      = req.body;
    const supplier = clientFor('supplier');
    const created = await supplier.create(
      T_INVOICE,
      {
        supplier: parties.supplier,
        buyer: parties.buyer,
        invoiceNumber,
        amount: String(amount),
        currency,
        dueDate,
        status: 'Draft',
      },
      `create-inv-${Date.now()}`,
    );
    res.json({
      contractId: created.contractId, invoiceNumber,
    });
  } catch (err) { next(err); }
});

app.post('/invoices/:cid/confirm', async (req, res, next) => {
  try {
    const tx = await clientFor('buyer').exercise(
      T_INVOICE, req.params.cid,
      'Confirm', {},
      `confirm-${Date.now()}`,
    );
    res.json({ updateId: tx.updateId });
  } catch (err) { next(err); }
});

app.get('/invoices', async (req, res, next) => {
  try {
    const role = req.query.as ?? 'supplier';
    const acs = await clientFor(role).activeContracts(T_INVOICE);
    res.json(acs);
  } catch (err) { next(err); }
});

app.listen(Number(process.env.PORT ?? 4000));
```

The repo has the corresponding routes for `/invoices/:cid/pay`, factoring offers, and the DvP settlement; each is the same six-line shape.

10.4 Running and Testing

In one terminal:

```
cd ~/canton-invoice-app/backend
```

```
npm start
```

Expect:

```
Resolved parties: {
  supplier: 'Supplier::1220f7b280be...',
  buyer:    'Buyer::1220b67d78e0...',
  factor:   'Factor::1220098a44bd...'
}
Invoice API listening on http://localhost:4000
```

In another, create an invoice, confirm it, list the ACS:

```
# Health
curl -s http://localhost:4000/health | jq

# Create (supplier)
curl -s -X POST http://localhost:4000/invoices \
  -H "Authorization: Bearer dev-secret" \
  -H "Content-Type: application/json" \
  -d '{
    "invoiceNumber":"API-100",
    "amount":"750.00",
    "currency":"USD",
    "dueDate":"2027-06-30T00:00:00Z"
  }'
# { "contractId":"...", "invoiceNumber":"API-100" }

# Confirm (buyer)
CID=<paste contractId from previous response>
curl -s -X POST \
  "http://localhost:4000/invoices/$CID/confirm" \
  -H "Authorization: Bearer dev-secret"
# { "updateId":"..." }

# List as supplier (should show one Confirmed invoice)
curl -s "http://localhost:4000/invoices?as=supplier" \
  -H "Authorization: Bearer dev-secret" | jq '.[0].contractEntry'
```

Change `?as=supplier` to `?as=factor` and you get back an empty list — the factor's participant never saw the invoice, which is the privacy story of Chapter 9 visible through the API.

Shared-secret auth is not real auth

`Bearer dev-secret` is fine for local development. Production requires a JWT signed by an issuer Canton trusts (`jwks-url` in the participant config), and the token must encode `actAs` entitlements. See Chapter 26 of the first book for the full pattern.

10.5 Why a Backend at All

A sophisticated frontend can speak to the JSON Ledger API directly. We still keep a backend for three reasons:

1. **Secrets.** The frontend should never hold API credentials, JWT signing keys, or issuer-private material. The backend does.
2. **Composition.** `POST /invoices` is atomic from the UI's point of view, but may eventually do more than one ledger command (emit a metric, publish to an audit log, trigger a notification). That belongs server-side.

3. **Shape.** The Ledger API returns protocol-level payloads. The backend reshapes them into the DTOs the UI actually wants — in the next chapter, `GET /invoices` becomes an array of plain `{id, status, amount}`, not a contract tree.

Chapter 11 builds that UI.

Chapter 11

The Frontend

The backend speaks plain HTTP. The frontend is React + Vite, plus one `fetch` helper and one component tree. No state manager, no routing library, no design system — we keep the dependency list to five packages so you can read the whole thing in an evening.

11.1 Shape

```
canton-invoice-app/
  frontend/
    package.json
    vite.config.js
    index.html
    src/
      main.jsx
      App.jsx
      api.js
```

```
cd ~/canton-invoice-app
mkdir -p frontend/src
cd frontend
```

Create `package.json`:

```
{
  "name": "canton-invoice-frontend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.3.4",
    "vite": "^6.0.5"
  }
}
```

Install:

```
npm install
```

11.2 Vite Config

We proxy `/api/*` to the backend so the browser can avoid CORS and the code never hardcodes `localhost:4000`.

```
// vite.config.js
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:4000',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, ''),
      },
    },
  },
});
```

11.3 The `fetch` Wrapper

One module, one responsibility. Every API call goes through `request`, which attaches the auth header and parses JSON.

```
// src/api.js
const BASE = '/api';
const TOKEN = 'dev-secret';

async function request(path, init = {}) {
  const res = await fetch(`${BASE}${path}`, {
    ...init,
    headers: {
      'Authorization': `Bearer ${TOKEN}`,
      'Content-Type': 'application/json',
      ...(init.headers ?? {}),
    },
  });
  const text = await res.text();
  const body = text ? JSON.parse(text) : null;
  if (!res.ok) {
    throw new Error(body?.error ?? `HTTP ${res.status}`);
  }
  return body;
}

export const api = {
  health:         () => request('/health'),
  listInvoices:   (role) => request(`/invoices?as=${role}`),
  createInvoice:  (payload) => request('/invoices',
    { method: 'POST', body: JSON.stringify(payload) }),
  confirmInvoice: (cid) => request(
    `/invoices/${encodeURIComponent(cid)}/confirm`,
    { method: 'POST' }),
```

```
  payInvoice:     (cid) => request(
    `/invoices/${encodeURIComponent(cid)}/pay`,
    { method: 'POST' }),
};
```

Bearer dev-secret is shipped in the bundle

The frontend is a public artefact. Baking `dev-secret` into the JS is fine locally but a credential leak in production. Real deployments mint short-lived JWTs on a user-login endpoint and store them in memory — see the reference book Chapter 15.

11.4 The App

`App.jsx` is one component. Three things matter:

1. A **role selector** (supplier / buyer / factor) that drives which `?as=` the list call uses.
2. A **create form** that only appears for the supplier.
3. A **list** that polls every three seconds; rows get `Confirm` / `Pay` buttons based on the viewer's role and the invoice status.

```
// src/App.jsx (excerpt)
import { useEffect, useState } from 'react';
import { api } from './api.js';

const ROLES = ['supplier', 'buyer', 'factor'];

function extractInvoice(entry) {
  const created =
    entry?.contractEntry?.JsActiveContract?.createdEvent;
  if (!created) return null;
  const arg = created.createArgument ?? {};
  return {
    contractId: created.contractId,
    invoiceNumber: arg.invoiceNumber,
    amount: arg.amount,
    currency: arg.currency,
    status: arg.status,
  };
}

export default function App() {
  const [role, setRole] = useState('supplier');
  const [invoices, setInvoices] = useState([]);

  async function refresh() {
    const data = await api.listInvoices(role);
    setInvoices(
      (Array.isArray(data) ? data : [])
        .map(extractInvoice).filter(Boolean),
    );
  }

  useEffect(() => {
    refresh();
    const t = setInterval(refresh, 3000);
    return () => clearInterval(t);
```

```
  }, [role]);

  // ... render role switcher, create form, list ...
}
```

Invoices returned from `/invoices` are wrapped in the Ledger API's `JsActiveContract` envelope. `extractInvoice` flattens that structure; once you do the extraction once, the rest of the UI works with plain `{contractId, status, amount}`.

11.5 Running the Stack

Three terminals:

```
# 1. Canton (already running from Part I)
cd ~/canton-invoice-app
canton daemon -c canton/network.conf \
  --bootstrap canton/bootstrap.canton

# 2. Backend
cd ~/canton-invoice-app/backend
npm start

# 3. Frontend
cd ~/canton-invoice-app/frontend
npm run dev
```

Open `http://localhost:5173`. You should see the role selector and an empty table.

11.6 Manual Test Walk-through

1. Stay on the *supplier* view. Fill in an invoice number (`INV-001`), amount (`1000.00`), currency (`USD`), due date. Click *Create invoice*. The table populates within one poll interval with a `Draft` row.
2. Click the *buyer* button. The same invoice is listed, because buyer is an observer on the template. The row has a *Confirm* button.
3. Click *Confirm*. Status flips to `Confirmed`. Now a *Pay* button appears.
4. Click *factor*. The list is empty. The factor is not a stakeholder of `Invoice`, so the factor's participant has never replicated it. Privacy is real.
5. Back on buyer, click *Pay*. Status flips to `Paid`. Supplier and buyer see the updated row; factor still sees nothing.

11.7 Building for Production

```
cd ~/canton-invoice-app/frontend
npm run build
```

Expect something like:

```
vite v6.4.2 building for production...
29 modules transformed.
dist/index.html                   0.33 kB | gzip:  0.24 kB
dist/assets/index-CvZRmhJ1.js  198.60 kB | gzip: 62.26 kB
built in 326ms
```

Serve `dist/` from any static host. The `/api` proxy is a dev-only convenience; in production, put the frontend and backend behind the same reverse proxy (Nginx, Traefik, Cloudflare), or point `BASE` in `api.js` at the backend's public URL.

Chapter 12 puts the pieces together: one-command boot, full lifecycle demo, and the scripts that make the whole thing repeatable.

Chapter 12

End to End

Eleven chapters in, every piece has been running on its own at least once. This chapter gives you the scripts and the demo to run the full stack in three commands and walk a client through the entire invoice-factoring lifecycle in the UI.

12.1 One-Command Network

Stop whatever you have running (Canton, PostgreSQL, backend, frontend). Save the script below as `scripts/start-network.sh`:

```
#!/bin/bash
# scripts/start-network.sh
set -e
cd "$(dirname "$0")/.."

echo "-> Checking port 5432 for non-Docker listeners..."
if lsof -iTCP:5432 -sTCP:LISTEN -nP 2>/dev/null \
    | awk 'NR>1 {print $1}' \
    | grep -vqi "com.docke\|docker"; then
  echo "ERROR: non-Docker process on localhost:5432."
  echo "  Canton will hit it instead of Docker Postgres and fail with"
  echo "  'role \"canton\" does not exist'. Usually native Postgres:"
  echo "    brew services stop postgresql@18"
  exit 1
fi

echo "-> Starting PostgreSQL..."
docker compose up -d >/dev/null

echo "-> Waiting for PostgreSQL..."
for i in {1..30}; do
  if PGPASSWORD=canton psql -h localhost -U canton -d canton \
      -tAc 'SELECT 1' >/dev/null 2>&1; then break; fi
  sleep 1
done
PGPASSWORD=canton psql -h localhost -U canton -d canton \
    -tAc 'SELECT 1' >/dev/null 2>&1 \
  || { echo "Postgres not reachable as 'canton'. If you booted the"; \
       echo "  volume before the init script existed, reset it:"; \
       echo "    docker compose down -v && docker compose up -d"; \
       exit 1; }

if lsof -i :5001 -sTCP:LISTEN -t >/dev/null 2>&1; then
  echo "-> Canton already running, skipping."
else
  mkdir -p log
```

```
  rm -f log/canton.log log/canton-stdout.log
  nohup canton daemon \
    -c canton/network.conf \
    --bootstrap canton/bootstrap.canton \
    --log-file-name log/canton.log \
    > log/canton-stdout.log 2>&1 &

  echo "-> Waiting for Canton Ledger APIs..."
  for i in {1..60}; do
    if lsof -i :5013 -sTCP:LISTEN -t >/dev/null 2>&1; then
      break
    fi
    sleep 1
  done

  # Ledger API binds before bootstrap finishes. Wait for the
  # bootstrap script to print "Network ready!" (or crash).
  echo "-> Waiting for bootstrap to complete..."
  for i in {1..120}; do
    if grep -q "Network ready!" log/canton-stdout.log 2>/dev/null; then
      break
    fi
    if grep -q "Bootstrap script terminated" \
        log/canton-stdout.log 2>/dev/null; then
      echo "ERROR: bootstrap crashed. Tail of log:"
      tail -10 log/canton-stdout.log
      echo "Likely stale Postgres volume. Reset with:"
      echo "  ./scripts/stop-network.sh --wipe"
      exit 1
    fi
    sleep 1
  done
fi

echo "-> Network ready."
```

Why the port 5432 preflight

On macOS the Docker publish `5432:5432` binds the wildcard address. If Homebrew already has `postgresql@18` running, it owns `localhost:5432` with a more specific bind and wins every connection. Canton then talks to your native Postgres (which has no `canton` role) and dies with `role "canton" does not exist`. The preflight stops the script before Canton touches the wrong database.

And the companion `scripts/stop-network.sh`:

```
#!/bin/bash
# scripts/stop-network.sh
# Pass --wipe to also delete the Postgres volume (full reset).
set -e
cd "$(dirname "$0")/.."
PIDS=$(lsof -ti :5001 2>/dev/null || true)
[ -n "$PIDS" ] && kill $PIDS 2>/dev/null || true

if [ "$1" = "--wipe" ]; then
  docker compose down -v >/dev/null
  echo "-> Stopped (volume wiped)."
else
  docker compose down >/dev/null
  echo "-> Stopped (volume preserved; use --wipe for fresh bootstrap)."
fi
```

Make both executable:

```
chmod +x scripts/start-network.sh scripts/stop-network.sh
```

12.2 Three-Terminal Boot

Terminal 1. Network + ledger:

```
cd ~/canton-invoice-app
./scripts/start-network.sh
```

The bootstrap is fully idempotent. `bootstrap.synchronizer` skips when a synchronizer with that name already exists, and the `enableIfMissing` helper from Chapter 5 looks up each party on its participant first and only calls `parties.enable` when it is absent — so restarts reuse existing parties instead of failing with `TOPOLOGY_MAPPING_ALREADY_EXISTS`. Plain `stop-network.sh` preserves the PostgreSQL volume so every restart picks up the same party IDs and ledger state; pass `-wipe` only when you want a fresh bootstrap from scratch. The start script still detects bootstrap crashes (it grep's the Canton log for `Bootstrap script terminated`) and points you at `-wipe` as a recovery step.

Terminal 2. Backend:

```
cd ~/canton-invoice-app/backend
npm start
# Invoice API listening on http://localhost:4000
```

Terminal 3. Frontend:

```
cd ~/canton-invoice-app/frontend
npm run dev
# Local:   http://localhost:5173/
```

Open `http://localhost:5173`.

12.3 The Demo Script

With everything running, a client demo is one uninterrupted narrative:

1. **Context.** "There are three businesses on this network. Each has its own Canton participant node. They share one synchronizer." Switch between *supplier*, *buyer*, and *factor* buttons to make the point visually — each view is a different participant's perspective on the same ledger.

2. **Raise an invoice.** On the supplier view, create `INV-2026-01` for `$12,500` due `2027-06-30`. Within the next poll, the row appears as `Draft`.

3. **Buyer receives it.** Click *buyer*. The same invoice is in the list (buyer is an observer of `Invoice`). The *Confirm* button is visible.

4. **Buyer confirms.** Click *Confirm*. The row flips to `Confirmed`; *Pay* appears. Behind the scenes we just exercised the `Confirm` choice on the supplier's `ContractId`.

5. **Factor offers.** In a real UI this is another form, hidden behind the *factor* tab; in this tutorial we are doing it by hand with `curl` (Chapter 10):

```
# Each Invoice choice (Confirm, Pay) archives the old
# contract and creates a new one --- so always fetch the
# CURRENT Confirmed contractId immediately before creating
# the FactoringOffer, or AcceptOffer will later fail with
# CONTRACT_NOT_FOUND on the stale invoiceCid.
CID=$(curl -s -H "Authorization: Bearer dev-secret" \
  "http://localhost:4000/invoices?as=supplier" \
  | jq -r '.[]
     | .contractEntry.JsActiveContract.createdEvent
     | select(.createArgument.invoiceNumber=="INV-2026-01"
       and .createArgument.status=="Confirmed")
     | .contractId')

SUPPLIER=$(curl -s \
  -H "Authorization: Bearer dev-secret" \
  http://localhost:4000/health \
  | jq -r '.parties.supplier')
BUYER=$(curl -s \
  -H "Authorization: Bearer dev-secret" \
  http://localhost:4000/health \
  | jq -r '.parties.buyer')
FACTOR=$(curl -s \
  -H "Authorization: Bearer dev-secret" \
  http://localhost:4000/health \
  | jq -r '.parties.factor')
# Send a direct JSON Ledger API call as factor.
# Capture response so we can pull the offer's contractId
# and templateId for the next step:
RESP=$(curl -s -X POST \
  http://localhost:5033/v2/commands/submit-and-wait-for-transaction-tree \
  -H "Content-Type: application/json" \
  -d '{"commands":[{"CreateCommand":{
        "templateId":"#invoice-app:Factoring:FactoringOffer",
        "createArguments":{
          "factor":"'"$FACTOR"'",
          "supplier":"'"$SUPPLIER"'",
          "buyer":"'"$BUYER"'",
          "invoiceCid":"'"$CID"'",
          "invoiceNumber":"INV-2026-01",
          "originalAmount":"12500.00",
          "currency":"USD",
          "dueDate":"2027-06-30T00:00:00Z",
          "discountRate":"0.05"}}}],
       "commandId":"offer-demo","userId":"ledger-api-user",
       "actAs":["'"$FACTOR"'"]}')
echo "$RESP" | jq .
```

6. **Supplier accepts.** Pull the offer's `contractId` and `templateId` out of `$RESP` with `jq`, then exercise `AcceptOffer` as supplier on port `5013`. The `templateId` is returned with a resolved 64-char package hash (not the `#invoice-app` shorthand), which is what the exercise endpoint needs:

```
OFFER_CID=$(echo "$RESP" | jq -r \
  '.transactionTree.eventsById."0"
    .CreatedTreeEvent.value.contractId')
OFFER_TID=$(echo "$RESP" | jq -r \
  '.transactionTree.eventsById."0"
    .CreatedTreeEvent.value.templateId')

curl -s -X POST \
  http://localhost:5013/v2/commands/submit-and-wait-for-transaction-tree \
```

```
  -H "Content-Type: application/json" \
  -d '{"commands":[{"ExerciseCommand":{
        "templateId":"'"$OFFER_TID"'",
        "contractId":"'"$OFFER_CID"'",
        "choice":"AcceptOffer",
        "choiceArgument":{}}}],
       "commandId":"accept-demo","userId":"ledger-api-user",
       "actAs":["'"$SUPPLIER"'"]}' | jq .
```

Don't line-wrap `contractId` in the payload

Canton contractIds are 136 characters. Inside a single-quoted `-d '{...}'` payload, a shell-wrapped line inserts a literal newline plus leading whitespace into the JSON string — Canton then replies with a plain-text error and `jq` dies with `parse error: Invalid numeric literal at line 1, column 8`. Keep each `contractId` (and each shell-expanded `"$VAR"`) on one line, or feed the payload from a file via `-d @payload.json`.

The `Invoice` contract archives and a `FactoringAgreement` is created (supplier and factor as signatories, buyer as observer). Back in the UI, the *Invoice* row disappears from supplier and buyer; factor now holds a new `FactoringAgreement`.

7. **Buyer settles.** Issue a `PaymentToken` to the buyer, exercise `SettleWithToken` — same atomic transaction, token moves to factor, agreement flips to `Settled`.

The factoring and payment interactions in the UI are intentionally sparse in this book. Extending `App.jsx` with *Create offer*, *Accept offer*, and *Pay with token* forms is a 150-line exercise you already have every piece for — the backend API, the `fetch` wrapper, and the polling list.

12.4 What You Have Built

At this point the stack is:

- A Canton network on one machine: PostgreSQL, a BFT sequencer, a mediator, three participants, all booted with one script.
- Three Daml modules (`Invoice`, `Factoring`, `Payment`) with fifteen passing tests plus a live-ledger smoke test.
- An Express backend that exposes HTTP routes for invoice lifecycle.
- A React frontend that lets a supplier, buyer, or factor view and act on what they are entitled to see.

The rest of the book zooms out. Chapter 13 shows how to replace our hand-rolled `PaymentToken` with the Canton Token Standard. Chapter 14 wires the running network into Prometheus, Grafana, and structured logs. Chapter 15 sketches the path from this laptop to a production deployment on the Global Synchronizer.

Part IV

Production Ready

Chapter 13

Wallet Integration

Our `PaymentToken` is fit for a textbook and nothing else. Every production payment asset on a Canton network uses the **Canton Token Standard** (CTS) — a set of Daml interfaces and workflows that define what a holding is, how it is issued, and how it is transferred. This chapter maps what we built onto what we would build, with references into the upstream documentation.

13.1 What the Token Standard Gives You

The standard decomposes a digital asset into three layers:

- **Instrument.** What the asset represents (e.g. "USD held at ACME Bank"). Owned by the issuer, versioned, interface-typed so non-issuer code can reason about it without knowing the concrete template.
- **Holding.** A specific party's stake in an instrument. Our `PaymentToken` is closest to this idea, but a real holding is an *interface* instance, not a single template — the issuer can upgrade the implementation without touching holder code.
- **Transfer factory.** A reusable workflow that mints, transfers, splits, and merges holdings according to the instrument's rules. Where our `Transfer` choice is one-shot (one current owner, one new owner), the standard's factory handles multi-leg atomic swaps, change-making, and fees.

The standard is formalized as CIP-56. The canonical introduction is the *canton.network* post "What is CIP-56? A Guide to Canton's Token Standard" (`canton.network/blog/what-is-cip-56-a-guide-t` the reference interfaces and implementation templates ship with the Daml SDK and are documented under "Canton Token Standard Overview" on `docs.daml.com`.

13.2 Mapping Our Model

Our Chapter 8 DvP transfers a `PaymentToken` inside the `SettleWithToken` choice. In CTS terms, the same settlement looks like:

1. The buyer holds an interface `HoldingCid : ContractId Holding` of some bank-issued USD instrument.
2. `SettleWithToken` takes an `instructionCid : ContractId TransferInstruction` — a pre-approved transfer that moves the amount from buyer to factor.

3. Inside the choice body, the transfer is finalized. The instrument's own rules decide whether a fee is taken, whether change is returned to the buyer if the holding was larger than the invoice, and so on.

In other words, the settlement logic moves out of our `FactoringAgreement` and into the instrument's transfer factory. Our choice body just validates amounts and kicks off the instruction. This keeps the factoring workflow agnostic to which currency (or stablecoin, or enterprise IOU) is being used for settlement.

13.3 Balances

The Ledger API's ACS query is enough for a one-participant balance. The CTS assumes a *wallet* service that aggregates holdings, computes a per-party balance, and tracks pending transfers. The reference implementation is the Canton *wallet backend*, published alongside the standard.

For our app we can get a rough balance by summing `PaymentToken.amount` for contracts where `owner` matches. Adapt `LedgerClient.activeContracts` — a one-liner in `src/routes/balance.js`:

```
// Rough balance query (sum of token amounts we own)
export async function balance(client, templateId) {
  const acs = await client.activeContracts(templateId);
  return acs.reduce((sum, entry) => {
    const arg = entry.contractEntry
      ?.JsActiveContract?.createdEvent?.createArgument;
    return sum + Number(arg?.amount ?? 0);
  }, 0);
}
```

This is not the CTS — it cannot tell you your balance across instruments, does not account for pending transfers, and ignores interface upgrades. It is a useful sanity check while iterating. For the real number, integrate the wallet backend and query it.

13.4 Further Reading

- CIP-56 introduction: `canton.network/blog/what-is-cip-56-a-guide-to-cantons-token-standard`. The motivation, layering (holding / instrument / transfer factory), and a walk-through of a transfer.
- Reference documentation: `docs.daml.com` → *Canton Token Standard Overview*. Interface definitions, implementation templates that ship with the SDK, and integration examples.
- CN-Quickstart (the cousin of this book that starts from a scaffolded template) wires a full wallet service with UI — useful for seeing the integration points we just described.

We leave the token upgrade as an exercise. The ledger logic in Chapters 6–8 is unaffected: `Invoice`, `FactoringOffer`, and `FactoringAgreement` keep their shape. What changes is the *kind* of contract passed to `SettleWithToken` — a `TransferInstruction` interface instance instead of our one-template `PaymentToken`.

Chapter 14

Monitoring Your Network

A Canton node that is running silently is a Canton node you cannot operate. This chapter wires the running network into Prometheus for metrics, points you at Grafana for dashboards, and covers the structured-logging story.

14.1 Turn On Metrics

Canton ships a Prometheus reporter. Append the following block to `canton/network.conf`:

```
canton {
  // ... sequencer/mediator/participant definitions ...

  monitoring.metrics {
    jvm-metrics.enabled = yes
    reporters = [
      {
        type = prometheus
        address = 0.0.0.0
        port = 9000
      }
    ]
  }
}
```

Restart with `./scripts/stop-network.sh && ./scripts/start-network.sh`. In production, restarting is a carefully orchestrated dance; locally it is a few seconds. Afterwards:

```
curl -s http://localhost:9000/metrics | head -20
```

```
# HELP daml_db_storage_general_executor_load Load of database pool
# TYPE daml_db_storage_general_executor_load gauge
daml_db_storage_general_executor_load{component="mediator",
  node="mediator1", query="query-tracking-disabled"} 0.0
daml_db_storage_general_executor_load{component="participant",
  node="participantSupplier", ...} 0.0
...
```

Count the families exposed:

```
curl -s http://localhost:9000/metrics | grep -c "^# HELP"
# 158
```

One endpoint, every node. The label `node=` carries the Canton node name, so a single scrape target collects data for the sequencer, mediator, and three participants.

14.2 What to Watch

The 158-strong list is a lot, and most of the labels name internal Canton components you will only care about when things are on fire. A practical starting set for day one:

- `daml_db_storage_general_executor_load` — the DB pool saturation. A node sustaining ≥ 0.8 under normal traffic is under-provisioned.
- `canton_parallel_indexer_inputs` and `canton_sequencer_client_event_handle_duration` — how fast a participant is consuming the sequencer. Rising means you are falling behind.
- `canton_mediator_confirmation_requests` — transactions per second at the mediator.
- JVM metrics (`process_cpu_seconds_total`, `jvm_memory_bytes_used`) — enabled because of `jvm-metrics.enabled = yes`. Same dashboard as any Java service.

14.3 Prometheus Scrape Config

A minimal `prometheus.yml` that treats Canton as one target:

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: canton
    static_configs:
      - targets: ['localhost:9000']
```

Run Prometheus alongside your other services:

```
docker run --rm -p 9090:9090 \
  -v $(pwd)/prometheus.yml:/etc/prometheus/prometheus.yml \
  prom/prometheus
```

Open `http://localhost:9090` and run, for example, this query:

```
rate(canton_mediator_confirmation_requests[5m])
```

14.4 Grafana

For dashboards, add Grafana to the same Compose stack:

```
# docker-compose.yml (snippet)
prometheus:
  image: prom/prometheus
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
  ports: ['9090:9090']
grafana:
  image: grafana/grafana:latest
  ports: ['3000:3000']
  environment:
    GF_SECURITY_ADMIN_PASSWORD: admin
```

At `http://localhost:3000`, log in as `admin/admin`, add Prometheus (`http://prometheus:9090`) as a data source, and start graphing. Digital Asset publishes a reference Canton dashboard JSON — search `canton-prometheus-dashboard` on GitHub.

Node health vs application health

Prometheus gives you *node* health — is the sequencer keeping up? Is the participant's DB saturated? It cannot tell you whether an invoice that should have been factored actually got factored. Application health requires *business* metrics: count of invoices in each state, time from `Draft` to `Confirmed`, discount rates over time. Add a small exporter in the backend that polls the ACS and publishes these under `invoice_...` names.

14.5 Structured Logging

Canton writes JSON-friendly log lines to `log/canton.log` by default. The file is high-volume and hard to read by eye; ship it to a log aggregator.

A local Loki + Promtail setup costs four lines of `docker-compose.yml` and lets you query logs in Grafana alongside the metrics. The upstream `grafana/promtail-config` has a Canton preset.

For this book we stop at "metrics enabled, logs on disk, nothing crashing". Chapter 15 points at the production-grade reading for the rest.

14.6 Alerting

Two alerts to set up on day one:

1. **DB pool saturation** — any `daml_db_storage_general_executor_load` > 0.9 for 5 minutes. Usually means the Postgres host is the bottleneck.

2. **Sequencer lag** — `canton_sequencer_client_event_handle_duration_seconds` > 1 for 5 minutes at any participant. The participant is falling behind the ordering layer; contracts it thinks are current may be stale.

Both are standard PromQL >-threshold alerts. Wire them to whatever you page on (PagerDuty, Opsgenie, Slack).

Canton will tell you, in great detail, what is wrong. Your job is to listen.

Chapter 15

What's Next

You have a running Canton network with a real multi-party application on top of it. From here, the distance to production is specific and well-trodden. This last chapter is a reading list — links and steps to get from your laptop to a live deployment.

15.1 From DevNet to the Canton Network

Our network uses one BFT sequencer and a private synchronizer called `invoice-network`. A production app talks to the **Canton Network** — a shared synchronizer operated by super-validators. The migration path:

1. Run a participant node against **DevNet** first (the public test network). The participant config points at the DevNet sequencer connection and drops the local sequencer/mediator definitions.
2. Apply for onboarding to **TestNet** or mainnet. Onboarding requires a cryptographic namespace handshake with the Super Validators.
3. Your application code is unchanged. Only the participant configuration (sequencer URL, TLS, JWT issuer) differs.

Digital Asset's `cn-quickstart` scaffolds a participant pre-wired for DevNet; use it as a reference.

15.2 Contract Upgrades

Our chapters bumped `version` every time we changed templates (`1.0.0 → 1.1.0 → 1.2.0 → 1.3.0`). That works because each upgrade was *additive* — new templates, new choices, never a changed field type.

For field-level upgrades:

```
dpm upgrade-check previous.dar next.dar
```

The tool tells you whether Canton will accept the new DAR as an upgrade. Breaking changes require a coordinated migration: deprecate the old template, offer a choice that creates the new template, then remove the old package after all live contracts have been migrated.

15.3 TLS and Auth in Production

Chapter 10 used `Bearer dev-secret`. The production story:

- **TLS on every connection** — participant Ledger API, participant admin API, sequencer public API, synchronizer bootstrap RPCs. The Canton distribution's `config/tls/` contains templates for mTLS between Canton components and server-side TLS for clients.
- **JWT-based authentication.** Participants accept signed JWTs whose payload encodes `actAs` and `readAs` entitlements. The frontend mints no tokens — the backend does, using an issuer that both it and the participant trust.
- **Party-level authorization.** Not every user should be able to submit as every party. Keep a mapping from user → party in the backend; hand the JWT to the Ledger API only after checking it.

Book 1 (*Daml & Canton: A Practical Guide*) Chapter 26 walks through a full TLS + JWT setup.

15.4 High Availability

Our topology had exactly one of each. For real deployments:

- **Multiple sequencers** in a BFT group. Clients (participants, mediators) are configured with a `sequencer-trust-threshold` and connect to a quorum. The Canton Open Source distribution ships with a `config/misc/bft-topology.conf` example.
- **Active-active participant** pairs sharing a Postgres instance — only one is master at a time; failover is automatic.
- **Synchronizer redundancy** via a second mediator with its own namespace, added to the topology after bootstrap.

15.5 Scaling

A single Canton participant on modest hardware processes low thousands of transactions per second. To scale:

1. Split the Postgres instance backing the participant onto its own server. Most load Canton generates is DB write pressure.
2. Add more participants for different parties. Contracts that only involve parties on participant A do not touch participant B.
3. For very high-throughput use cases, run the BFT sequencer group on dedicated hosts with fast disks — the sequencer is the global ordering layer and every transaction crosses it.

15.6 Observability Reading List

- *Canton Operations Manual* — the canonical reference for health checks, metrics, and log patterns.
- Grafana dashboards published under `digital-asset/canton-dashboards` on GitHub.
- `docs.daml.com` → *Monitoring* section.

15.7 What You Have Learned

In fifteen chapters we went from an empty directory to:

- A Canton network you booted by hand: Postgres, sequencer, mediator, three participants.
- A Daml application with three templates, interface-ready structure, and a tested factoring workflow.
- Live-ledger tests, a backend, a UI, a monitoring stack.

Nothing here used a shortcut distribution or a hidden quickstart. Every piece, you wrote. That is the only way to operate a distributed ledger: you need to know where every process came from, because when one misbehaves you will need to walk into its logs and understand what you see.

Happy factoring.